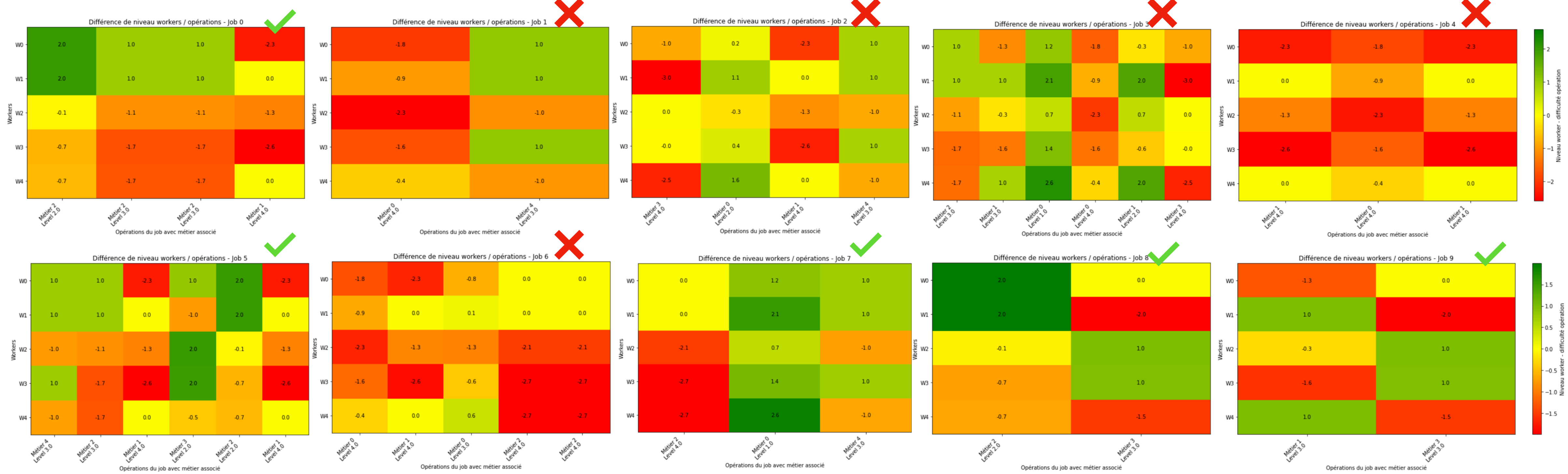
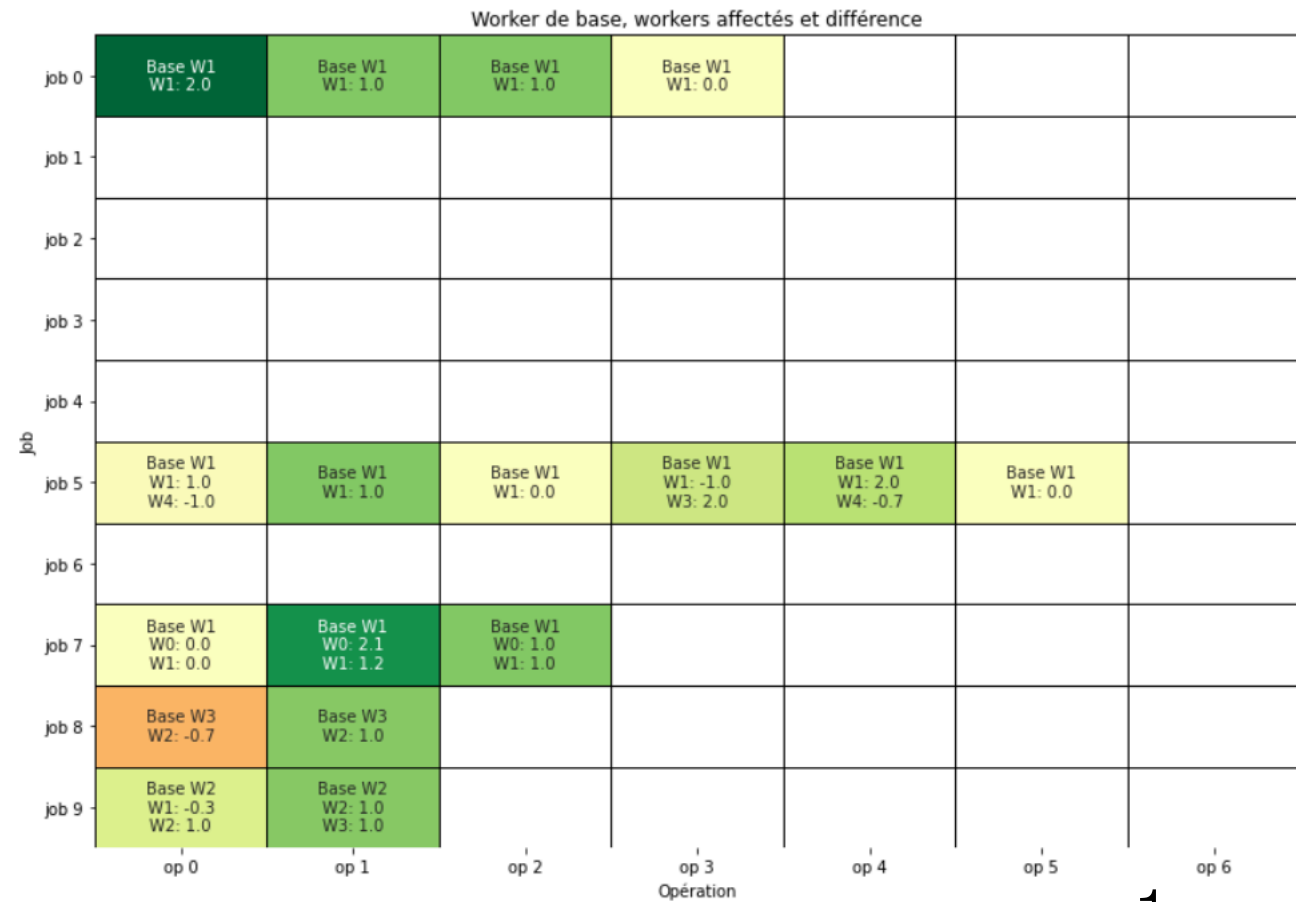
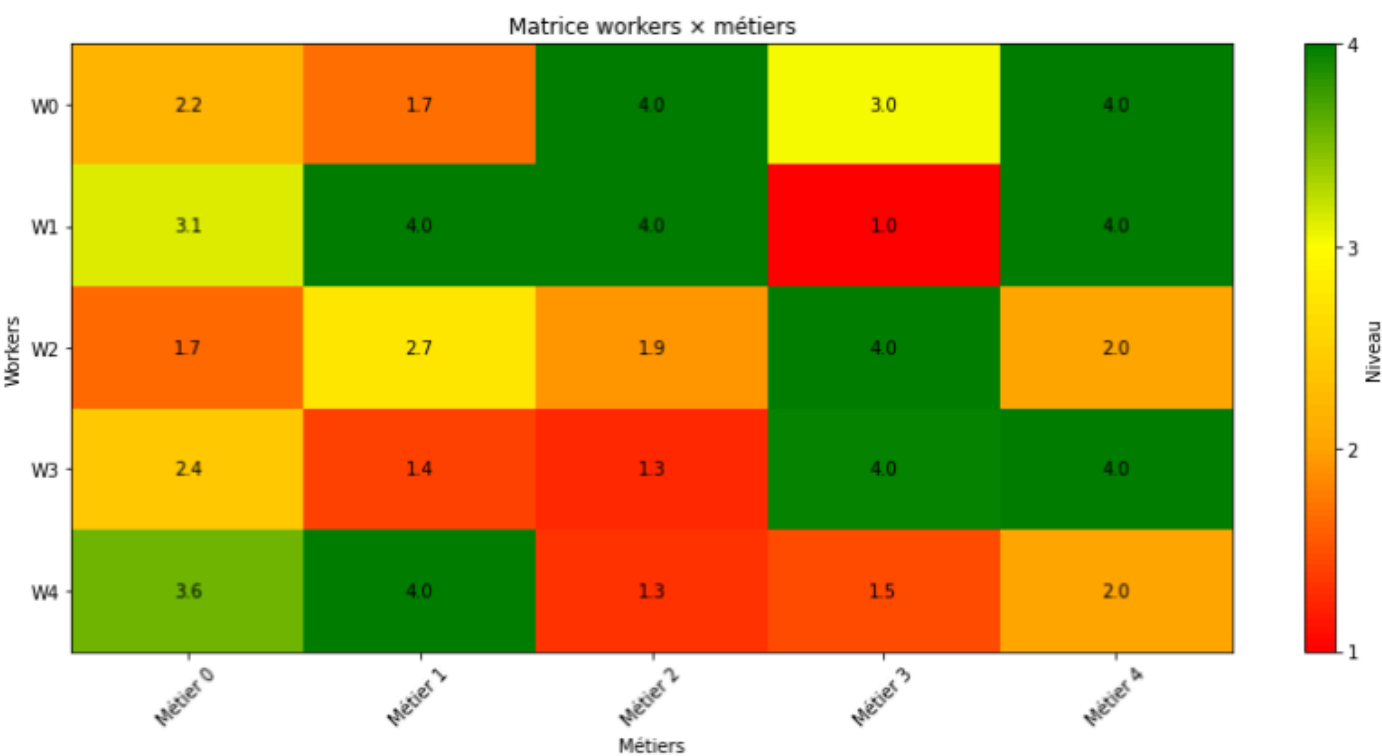


Visualisation instance

`instance.vizualize_differences_levels_workers_job_i(i)`

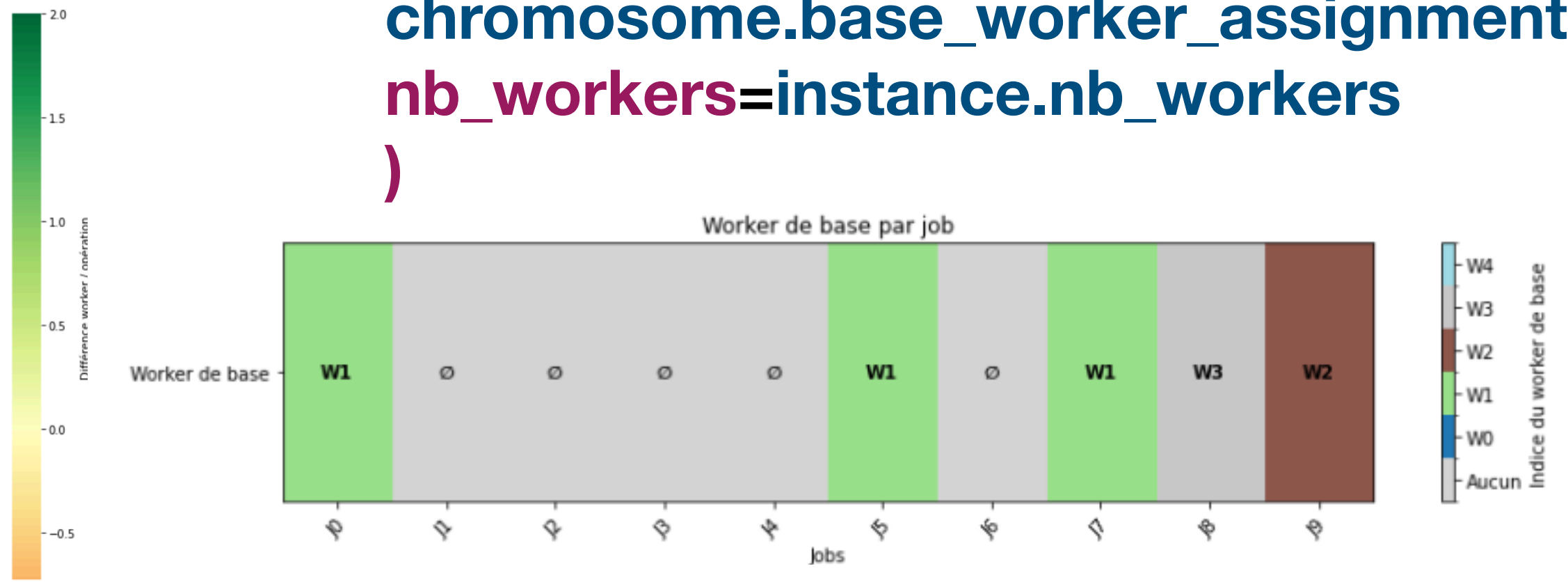


`instance.vizualize_levels_workers()`

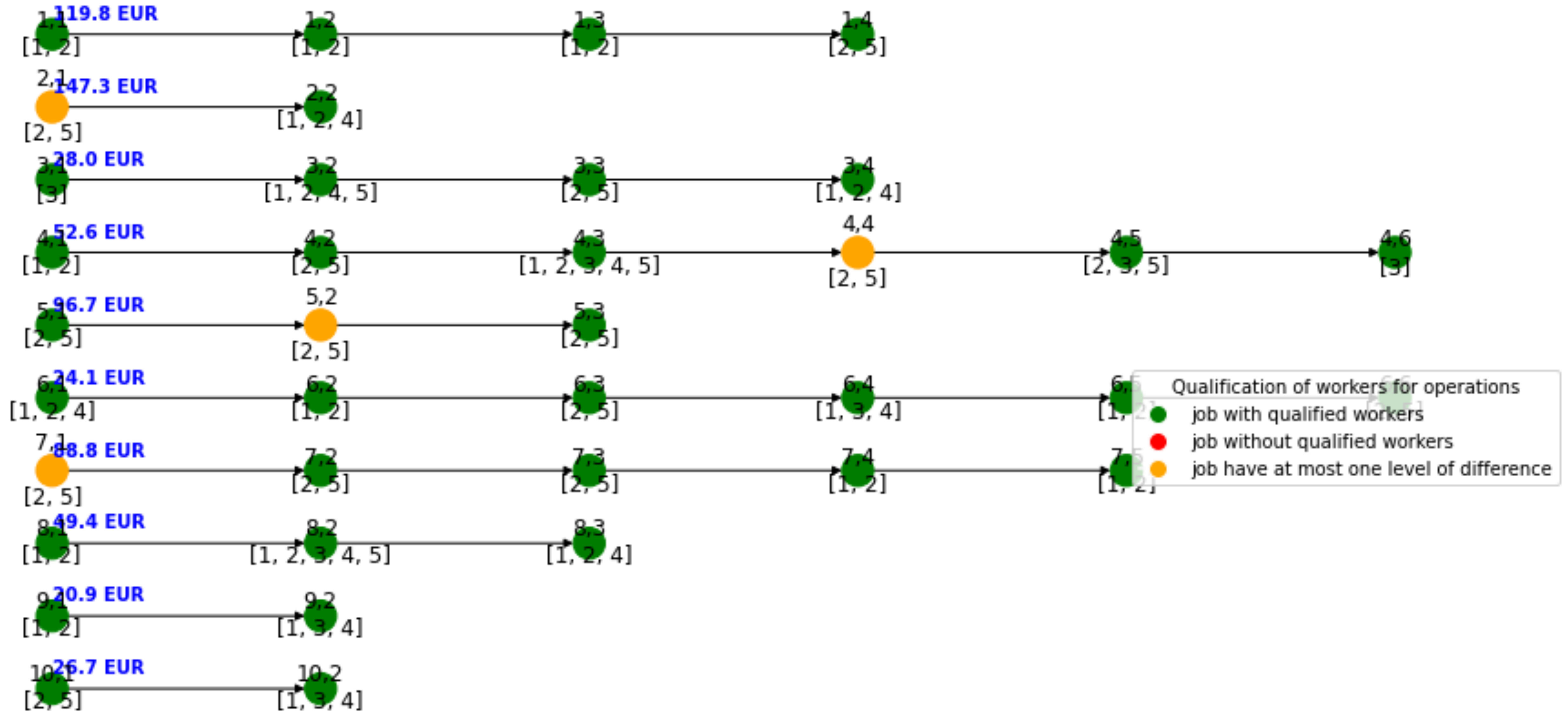


`visualize_base_worker_heatmap(`

`chromosome.base_worker_assignment,`
`nb_workers=instance.nb_workers`
`)`



Graphe de précedence de toutes les opérations de tous les jobs



Vue d'ensemble des Jobs - Opérations, métiers, Temps et Workers

Représentation instance
Plus enrichie

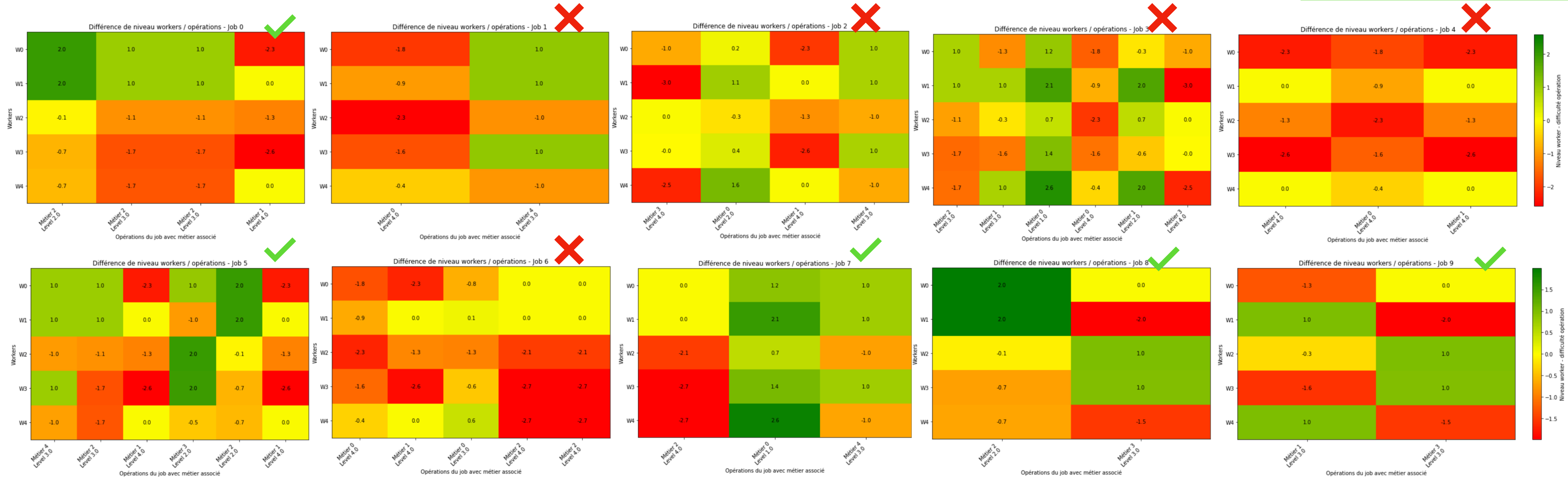
Job	Op 1	Op 2	Op 3	Op 4	Op 5	Op 6
J1	m3 Df=2	m3 Df=3	m3 Df=3	m2 Df=4		
	A:1.4 L:2.4 C:0.6	A:4.7 L:10.2 C:2.8	A:11.3 L:24.5 C:N/A	A:7.0 L:11.0 C:N/A		
	✓ w1 w2	✓ w1 w2	✓ w1 w2	✓ w2 w5		
	≈ w3 w4 w5	≈ -	≈ -	≈ -		
	✗ -	✗ w3 w4 w5	✗ w3 w4 w5	✗ w1 w3 w4		
J2	m1 Df=4	m5 Df=3				
	A:6.3 L:10.4 C:3.6	A:1.3 L:2.5 C:0.8				
	✓ -	✓ w1 w2 w4				
	≈ w2 w5	≈ w3 w5				
	✗ w1 w3 w4	✗ -				
J3	m4 Df=4	m1 Df=2	m2 Df=4	m5 Df=3		
	A:3.0 L:6.0 C:1.9	A:11.6 L:18.5 C:7.9	A:5.6 L:11.2 C:3.8	A:7.2 L:10.5 C:2.9		
	✓ w3	✓ w1 w2 w4 w5	✓ w2 w5	✓ w1 w2 w4		
	≈ w1 w4	≈ w3	≈ -	≈ w3 w5		
	✗ w2 w5	✗ -	✗ w1 w3 w4	✗ -		
J4	m3 Df=3	m2 Df=3	m1 Df=1	m1 Df=4	m2 Df=2	m4 Df=4
	A:2.2 L:3.4 C:1.7	A:6.8 L:11.9 C:4.4	A:3.2 L:5.3 C:2.5	A:6.3 L:10.4 C:3.6	A:8.2 L:12.3 C:3.4	A:3.8 L:6.5 C:N/A
	✓ w1 w2	✓ w2 w5	✓ w1 w2 w3 w4 w5	✓ -	✓ w2 w3 w5	✓ w3
	≈ -	≈ w3	≈ -	≈ w2 w5	≈ w1 w4	≈ w1 w4
	✗ w3 w4 w5	✗ w1 w4	✗ -	✗ w1 w3 w4	✗ -	✗ w2 w5
J5	m2 Df=4	m1 Df=4	m2 Df=4			
	A:6.4 L:12.6 C:2.8	A:8.0 L:11.4 C:4.4	A:5.6 L:11.2 C:3.8			
	✓ w2 w5	✓ -	✓ w2 w5			
	≈ -	≈ w2 w5	≈ -			

Job	Op 1	Op 2	Op 3	Op 4	Op 5	Op 6
J6	m5 Df=3	m3 Df=3	m2 Df=4	m4 Df=2	m3 Df=2	m2 Df=4
	A:3.8 L:8.0 C:1.5	A:2.9 L:5.0 C:2.3	A:3.4 L:7.0 C:1.6	A:3.8 L:6.2 C:2.2	A:1.4 L:2.4 C:0.6	A:6.4 L:12.6 C:2.8
	✓ w1 w2 w4	✓ w1 w2	✓ w2 w5	✓ w1 w3 w4	✓ w1 w2	✓ w2 w5
	≈ w3 w5	≈ -	≈ -	≈ w2 w5	≈ w3 w4 w5	≈ -
	✗ -	✗ w3 w4 w5	✗ w1 w3 w4	✗ -	✗ -	✗ w1 w3 w4
J7	m1 Df=4	m2 Df=4	m1 Df=3	m3 Df=4	m3 Df=4	
	A:4.0 L:5.8 C:N/A	A:1.3 L:2.8 C:0.6	A:4.2 L:7.8 C:2.1	A:9.1 L:14.3 C:3.8	A:6.7 L:13.2 C:4.0	
	✓ -	✓ w2 w5	✓ w2 w5	✓ w1 w2	✓ w1 w2	
	≈ w2 w5	≈ -	≈ w1 w4	≈ -	≈ -	
	✗ w1 w3 w4	✗ w1 w3 w4	✗ w3	✗ w3 w4 w5	✗ w3 w4 w5	
J8	m3 Df=4	m1 Df=1	m5 Df=3			
	A:6.2 L:10.0 C:4.8	A:3.2 L:5.3 C:2.5	A:1.3 L:2.0 C:0.7			
	✓ w1 w2	✓ w1 w2 w3 w4 w5	✓ w1 w2 w4			
	≈ -	≈ -	≈ w3 w5			
	✗ w3 w4 w5	✗ -	✗ -			
J9	m3 Df=2	m4 Df=3				
	A:4.5 L:9.7 C:N/A	A:2.6 L:5.1 C:N/A				
	✓ w1 w2	✓ w1 w3 w4				
	≈ w3 w4 w5	≈ -				
	✗ -	✗ w2 w5				
J10	m2 Df=3	m4 Df=3				
	A:6.8 L:11.3 C:4.1	A:10.7 L:18.0 C:6.3				
	✓ w2 w5	✓ w1 w3 w4				
	≈ w3	≈ -				
	✗ w1 w4	✗ w2 w5				

Difference de niveau entre chaque worker et chaque opération

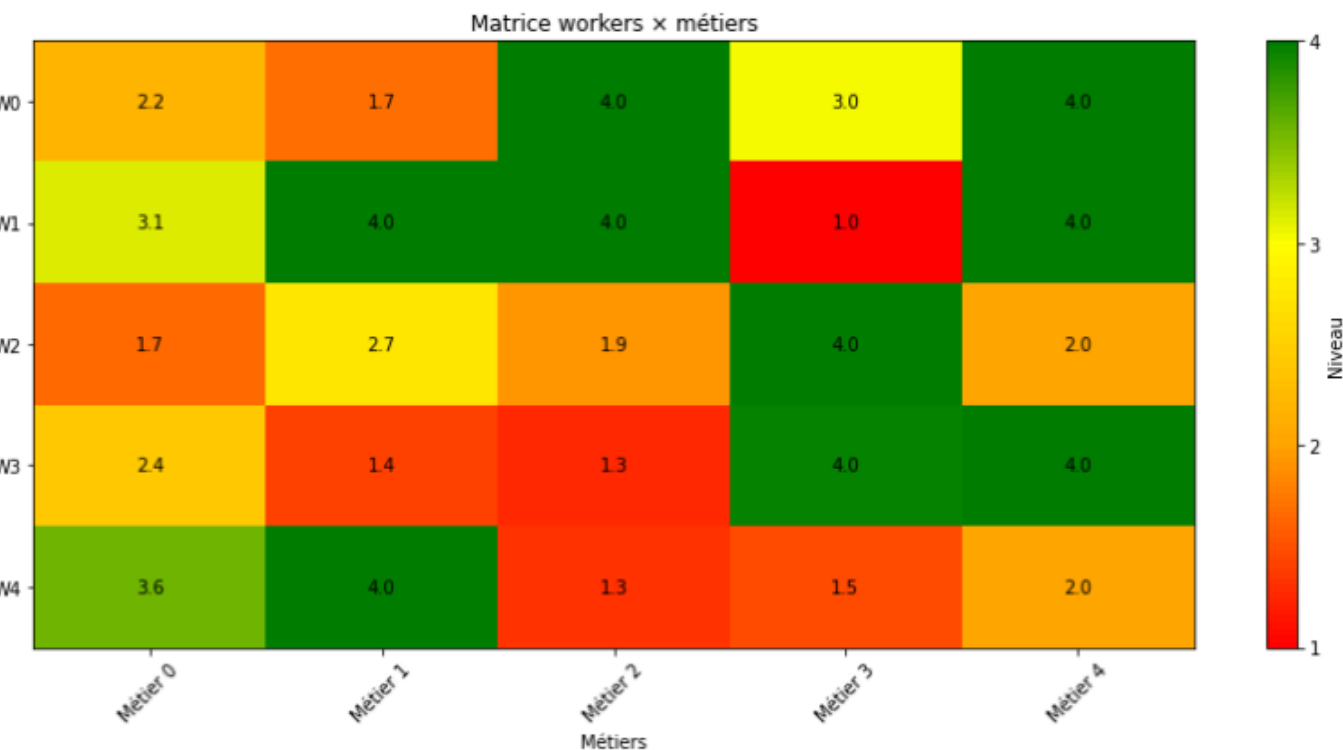
`instance.vizualize_differences_levels_workers_job(i)`

Représentation instance
Plus enrichie

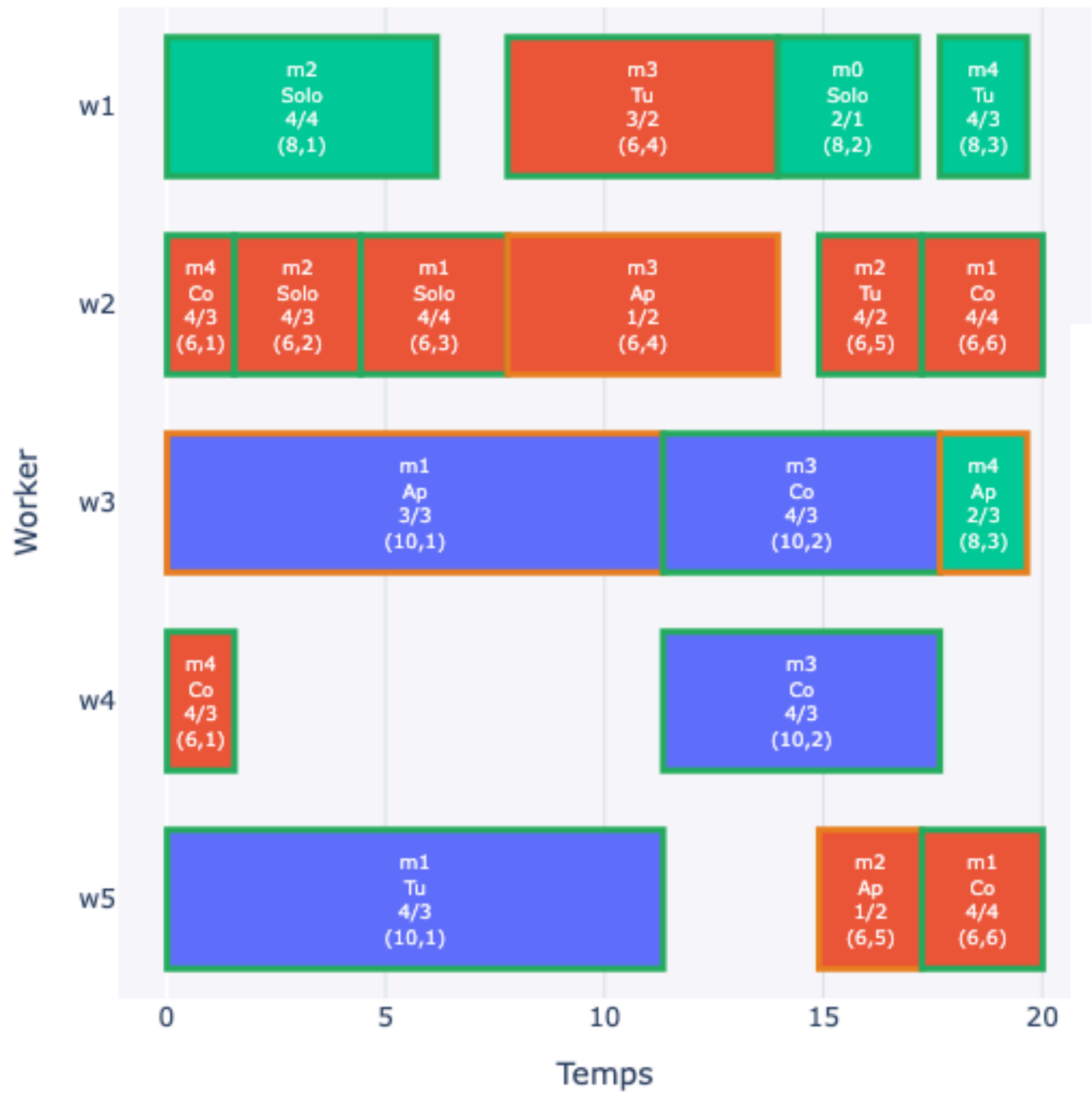


Niveau des workers par métier

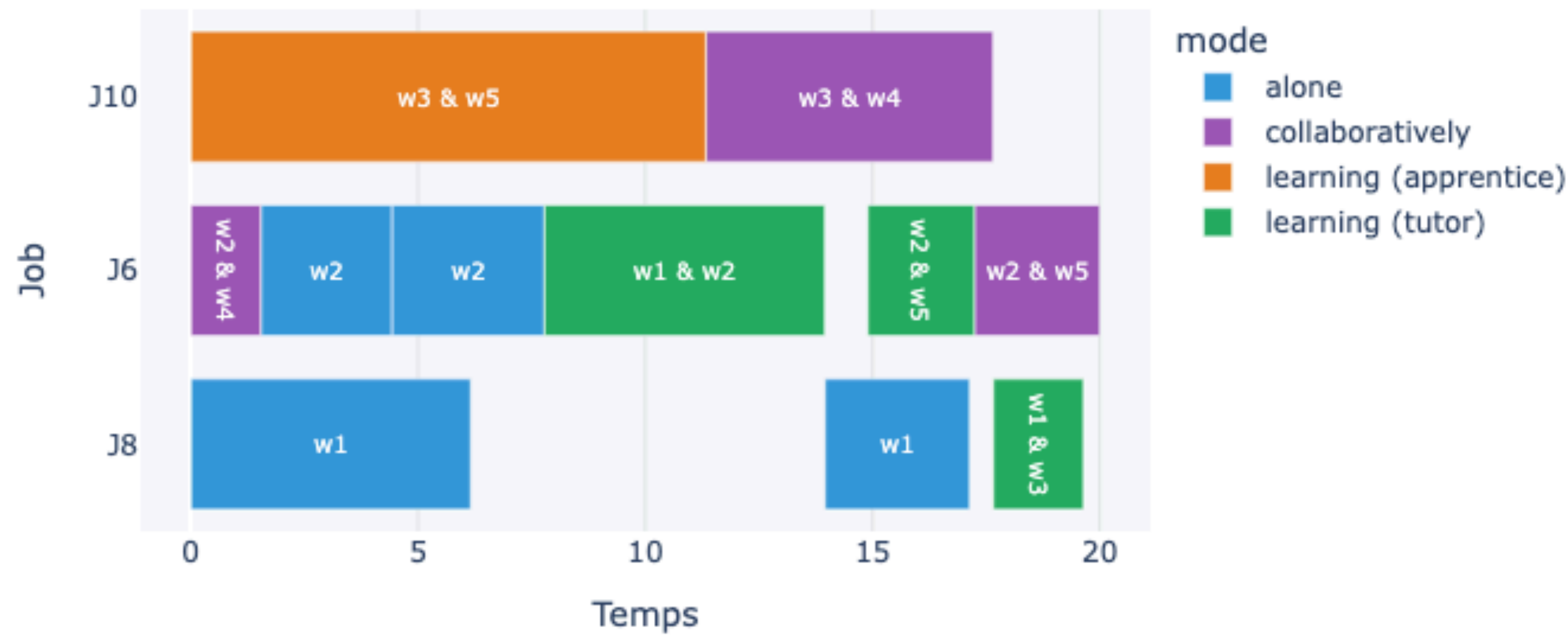
`instance.vizualize_levels_workers()`



MILP en résolution Lexicographique
Profit -> Skills -> Cognitive Load



Planning par Job (makespan = 20.00)



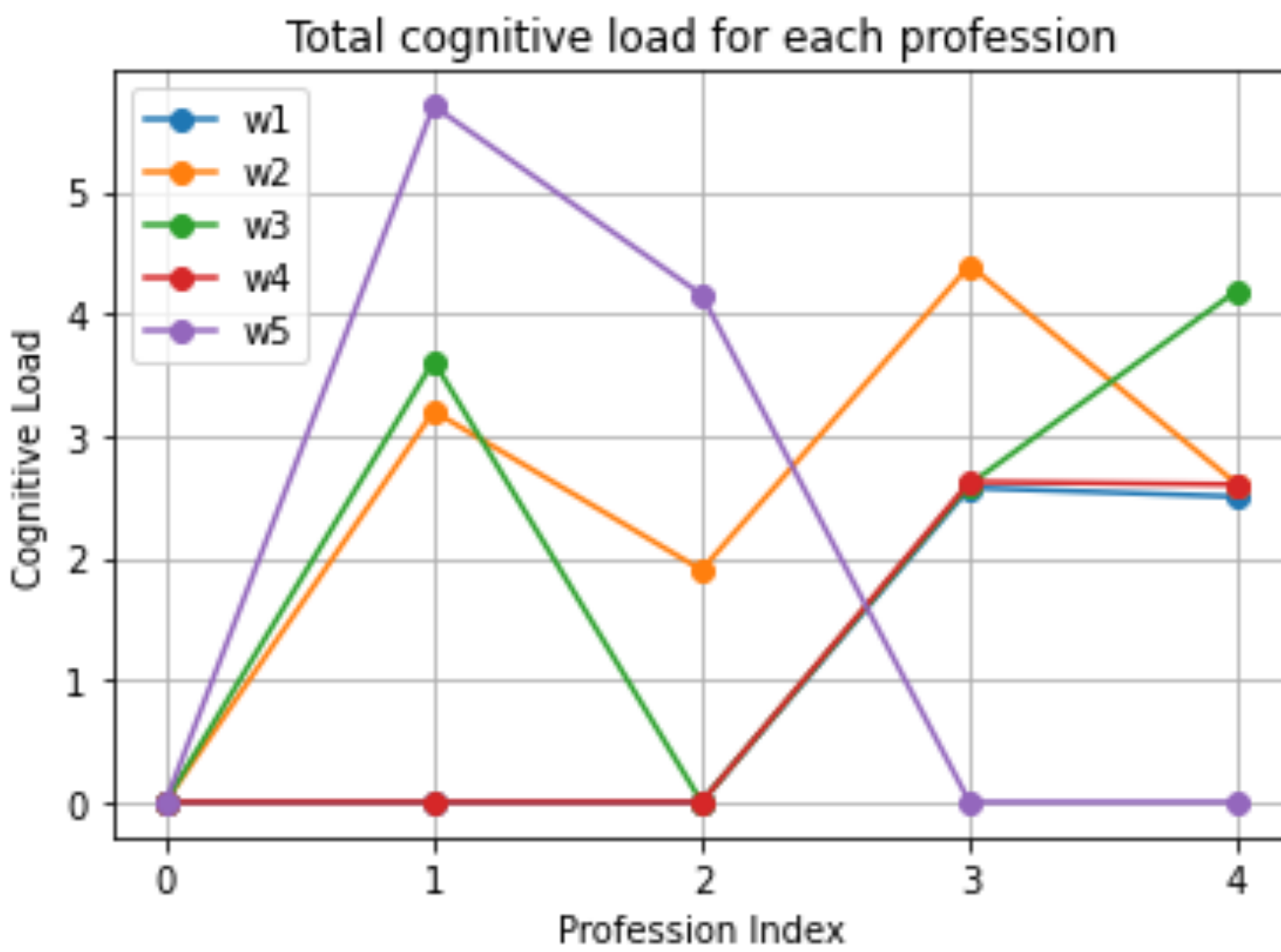
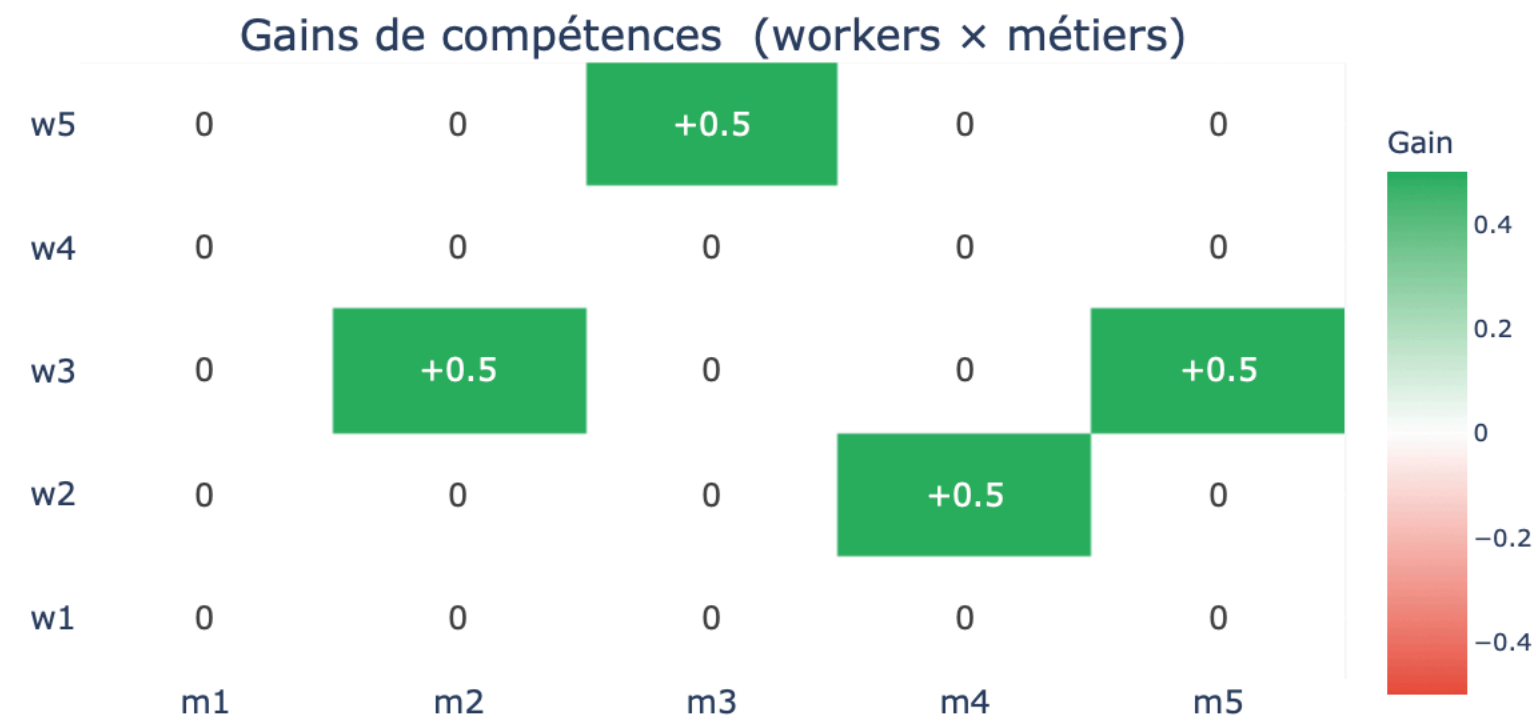
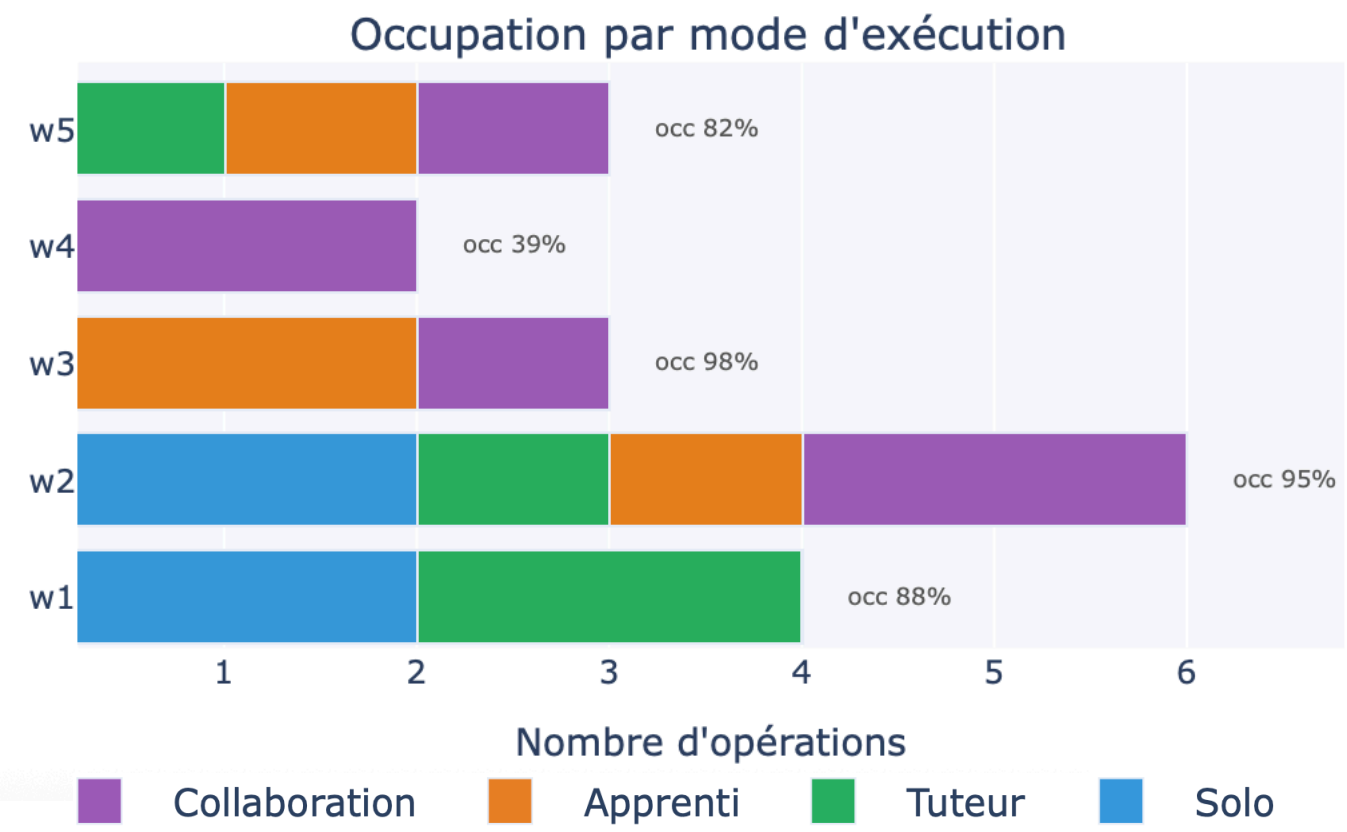
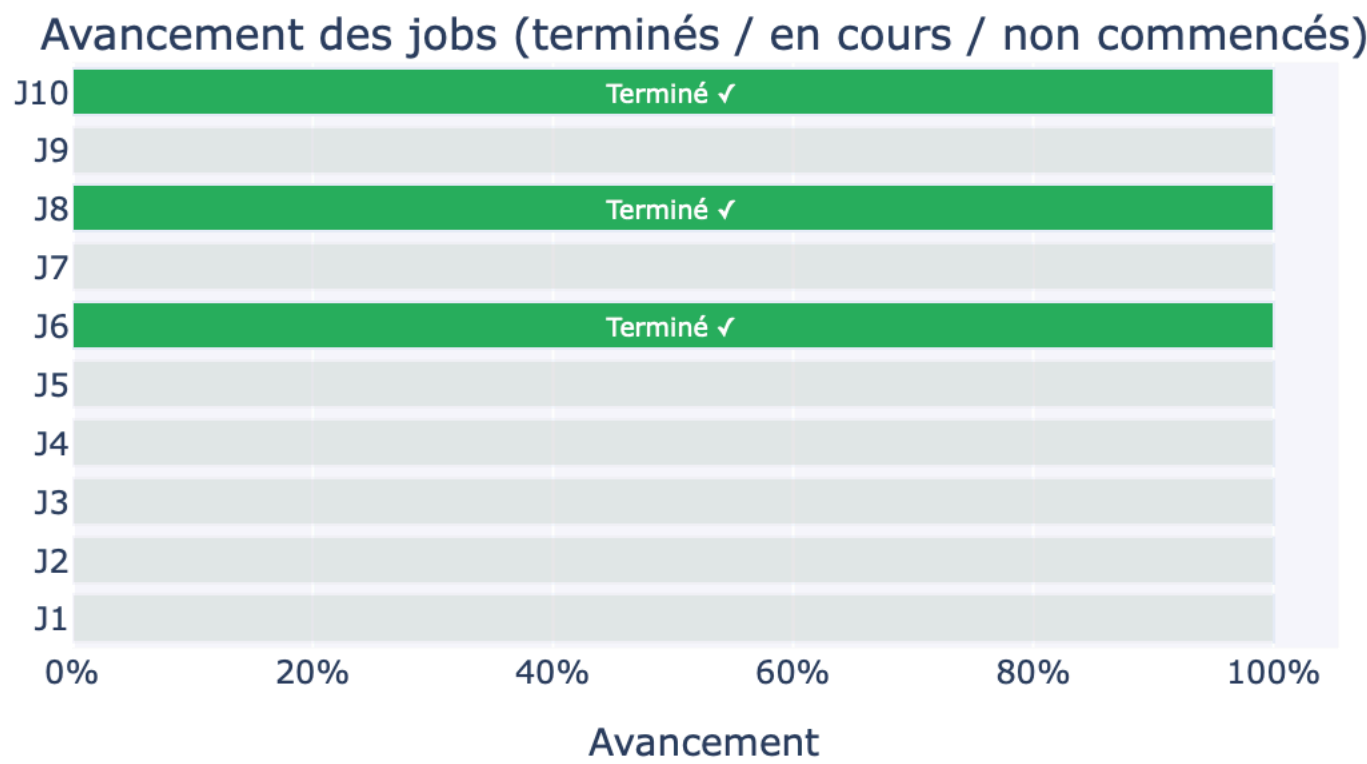
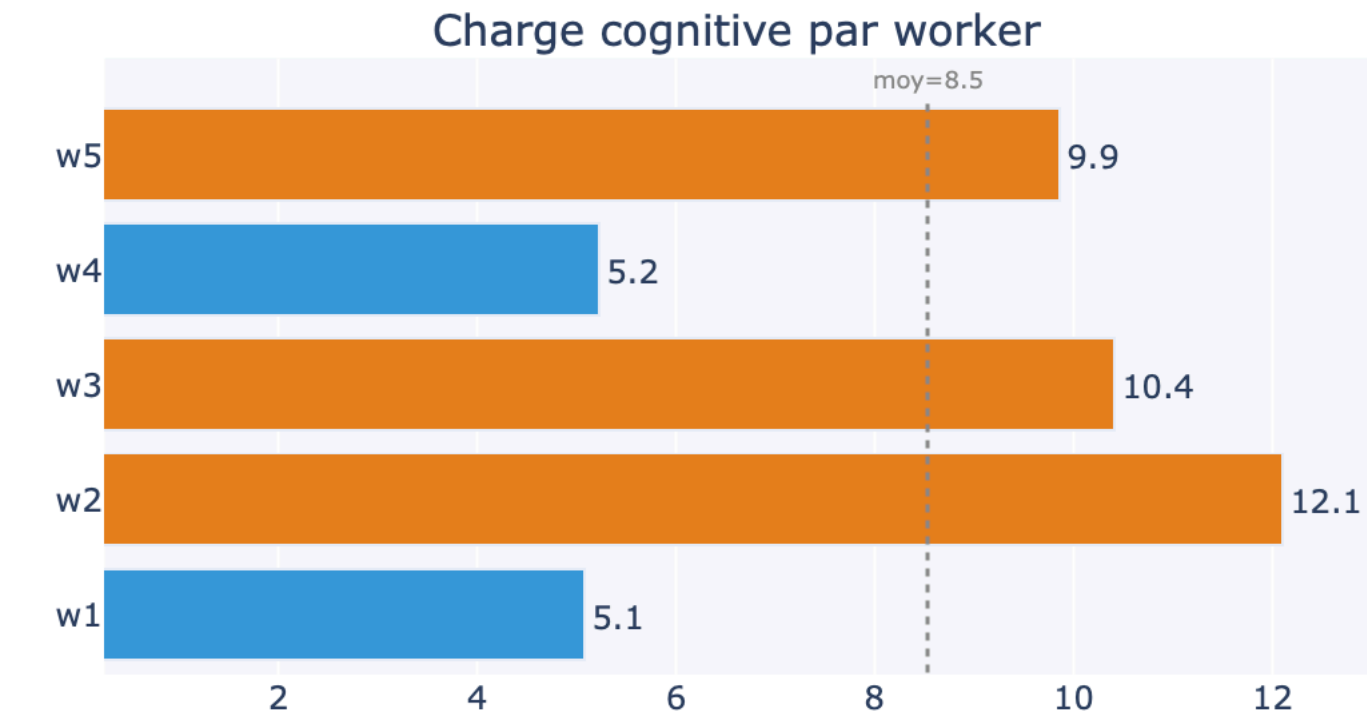
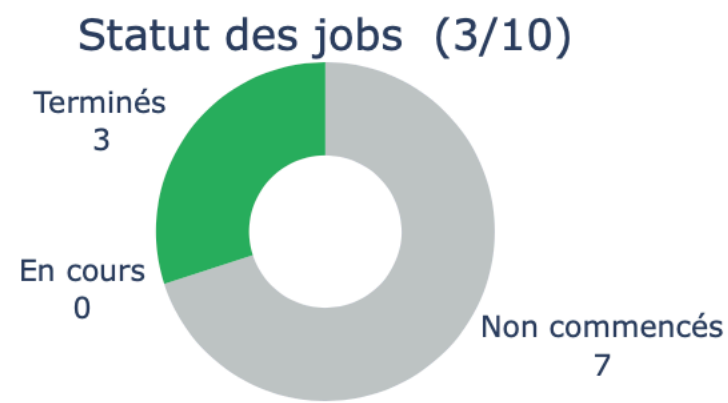
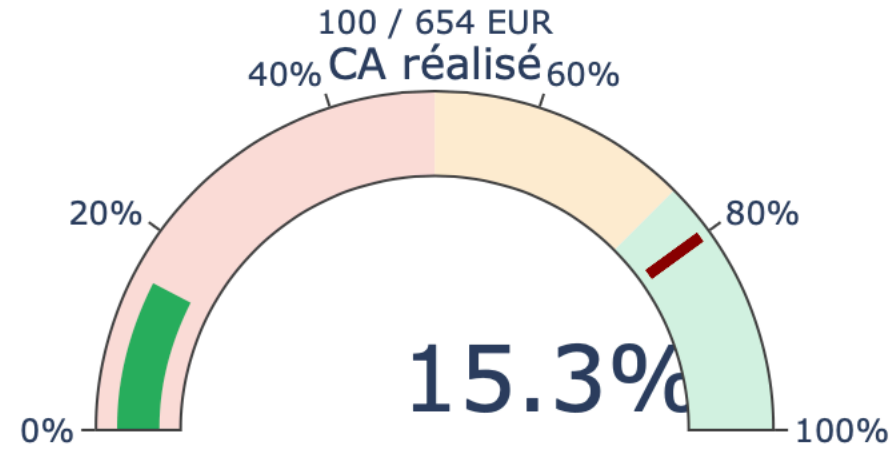
Exécution du test solution_valide...
Exécution du test base_worker...
Exécution du test need_level_if_solo...

	Nom	C_max	Profit_done	Skill_gain	Cognitive_load	solution_valide	base_worker	need_level_if_solo	OK
0	solution	20.0	111.221221	2.0	42.644282	PASS	PASS	PASS	TRUE

Ordonnancement passe les fonctions de test

MILP Résolution

Dashboard Business — Solution MILP | Makespan = 20.00 | Backlog = 0.0 u



Représentation en DataFrame

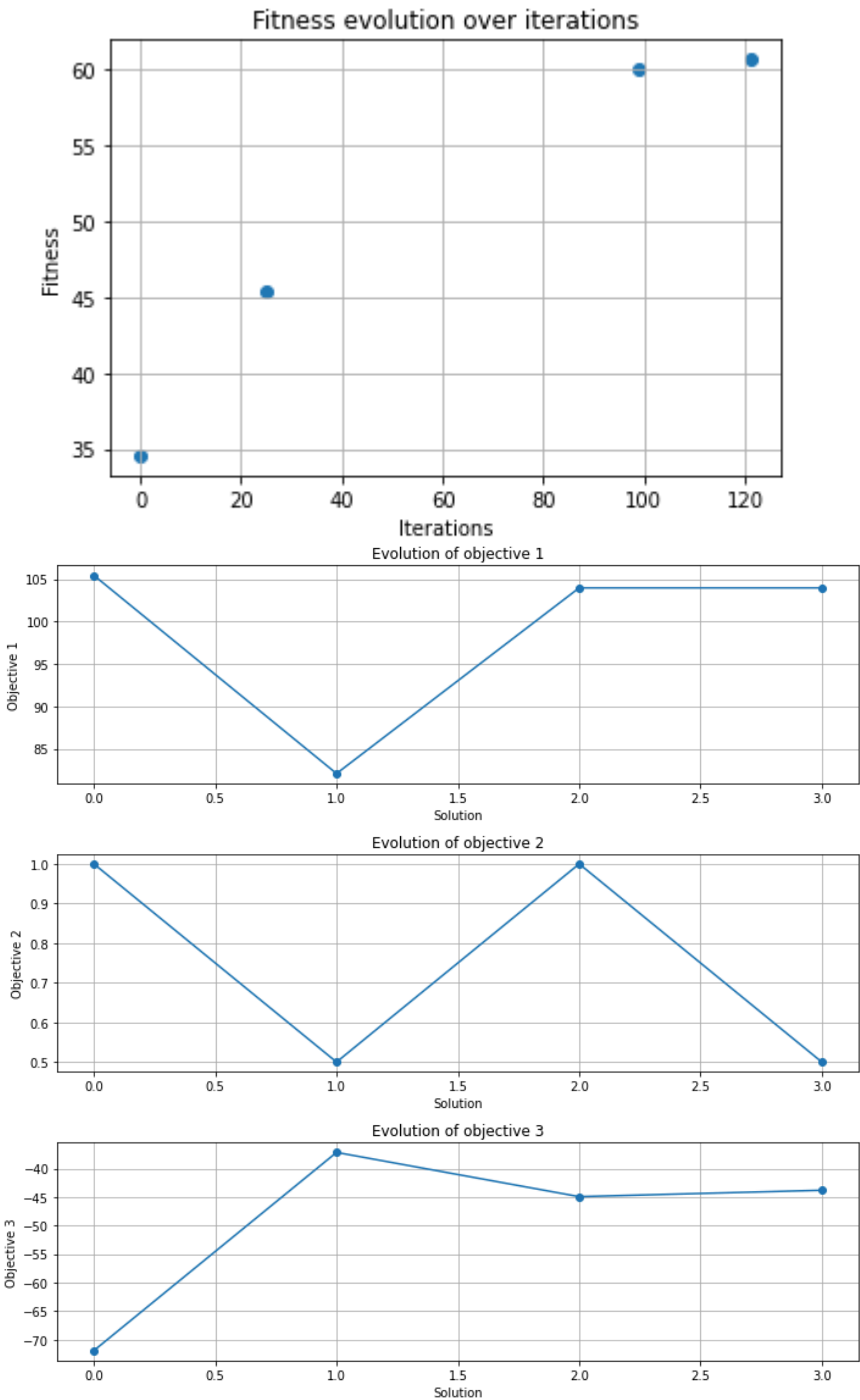
	Task	Start	Finish	Finish (var. f)	Processing time	Operation	Job	mode	Level_w	Difficulty_task	Job_done	Elementary task	Metier
0	w1	0.000000	6.156768	6.156768	6.156768	(8,1)	J8	alone	4.000000	4.0	True	38.0	2.0
1	w1	7.776647	13.948353	13.948353	6.171705	(6,4)	J6	learning (tutor)	3.038431	2.0	True	55.0	3.0
2	w1	13.948353	17.135557	17.135557	3.187204	(8,2)	J8	alone	2.175527	1.0	True	3.0	0.0
3	w1	17.650813	19.640491	19.640491	1.989678	(8,3)	J8	learning (tutor)	4.000000	3.0	True	73.0	4.0
4	w2	0.000000	1.540318	1.540318	1.540318	(6,1)	J6	collaboratively	4.000000	3.0	True	61.0	4.0
5	w2	1.540318	4.426609	4.426609	2.886291	(6,2)	J6	alone	4.000000	3.0	True	31.0	2.0
6	w2	4.426609	7.776647	7.776647	3.350038	(6,3)	J6	alone	4.000000	4.0	True	23.0	1.0
7	w2	7.776647	13.948353	13.948353	6.171705	(6,4)	J6	learning (apprentice)	1.007461	2.0	True	55.0	3.0
8	w2	14.888483	17.239974	17.239974	2.351491	(6,5)	J6	learning (tutor)	4.000000	2.0	True	44.0	2.0
9	w2	17.239974	20.000000	20.000000	2.760026	(6,6)	J6	collaboratively	4.000000	4.0	True	19.0	1.0
10	w3	0.000000	11.328800	11.328800	11.328800	(10,1)	J10	learning (apprentice)	2.736748	3.0	True	22.0	1.0
11	w3	11.328800	17.650813	17.650813	6.322013	(10,2)	J10	collaboratively	4.000000	3.0	True	53.0	3.0
12	w3	17.650813	19.640491	19.640491	1.989678	(8,3)	J8	learning (apprentice)	2.010535	3.0	True	73.0	4.0
13	w4	0.000000	1.540318	1.540318	1.540318	(6,1)	J6	collaboratively	4.000000	3.0	True	61.0	4.0
14	w4	11.328800	17.650813	17.650813	6.322013	(10,2)	J10	collaboratively	3.973705	3.0	True	53.0	3.0
15	w5	0.000000	11.328800	11.328800	11.328800	(10,1)	J10	learning (tutor)	4.000000	3.0	True	22.0	1.0
16	w5	14.888483	17.239974	17.239974	2.351491	(6,5)	J6	learning (apprentice)	1.307572	2.0	True	44.0	2.0
17	w5	17.239974	20.000000	20.000000	2.760026	(6,6)	J6	collaboratively	4.000000	4.0	True	19.0	1.0

Passe la fonction de test sur dataframe qui verifie bien les niveau des workers par rapport au niveau des opérations et qu'il n'y est pas d'incohérence

Random Search

Vérification que toutes les solution améliorante passe les testes

Random Search



```
1 for i in range(len(best)):
2     s_rs = dec.decode_solution(best[i])
3     row, ok = validate_model_solution(
4         instance=instance,
5         solution=s_rs,
6         constraints_config=constraints_config,
7     )
8     display(pd.DataFrame([row]))
```

✓ 0.2s

Exécution du test solution_valide...
Exécution du test base_worker...
Exécution du test need_level_if_solo...

	Nom	C_max	Profit_done	Skill_gain	Cognitive_load	solution_valide	base_worker	need_level_if_solo	OK
0	solution	20	105.439243	1.0	71.888733	PASS	PASS	PASS	TRUE

Exécution du test solution_valide...
Exécution du test base_worker...
Exécution du test need_level_if_solo...

	Nom	C_max	Profit_done	Skill_gain	Cognitive_load	solution_valide	base_worker	need_level_if_solo	OK
0	solution	20	82.090292	0.5	37.175307	PASS	PASS	PASS	TRUE

Exécution du test solution_valide...
Exécution du test base_worker...
Exécution du test need_level_if_solo...

	Nom	C_max	Profit_done	Skill_gain	Cognitive_load	solution_valide	base_worker	need_level_if_solo	OK
0	solution	20	103.970774	1.0	44.92481	PASS	PASS	PASS	TRUE

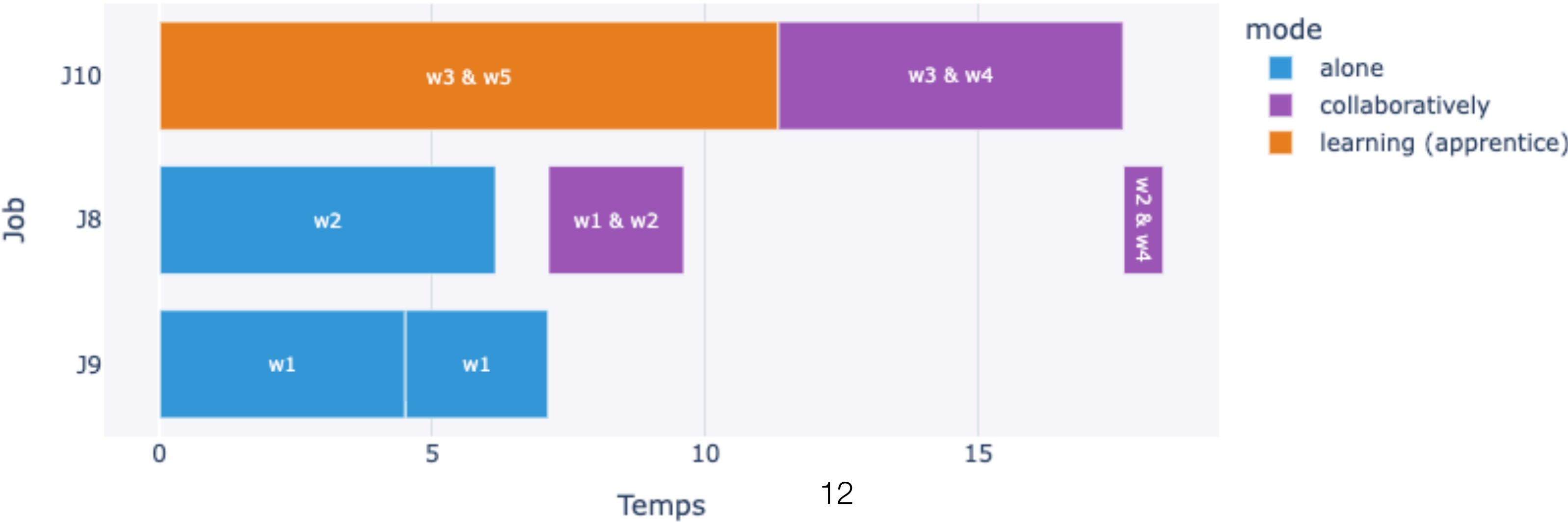
Exécution du test solution_valide...
Exécution du test base_worker...
Exécution du test need_level_if_solo...

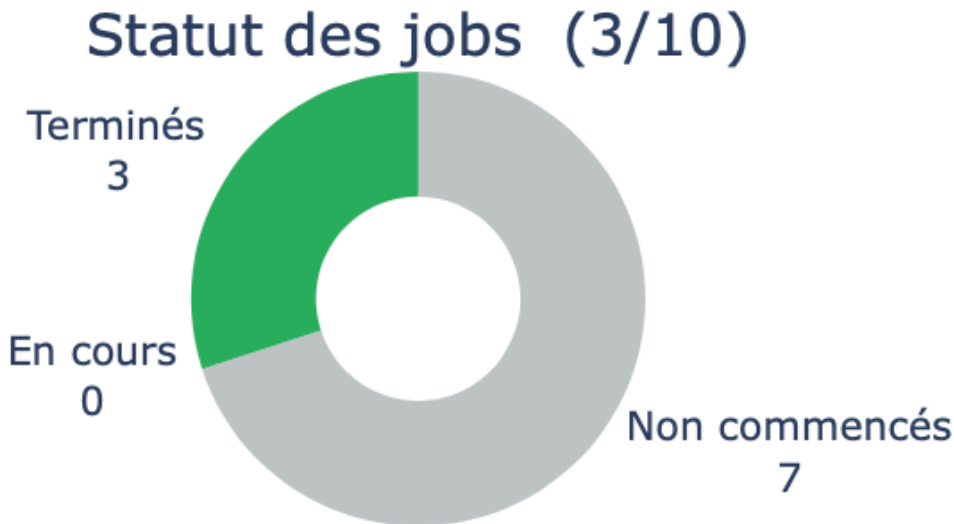
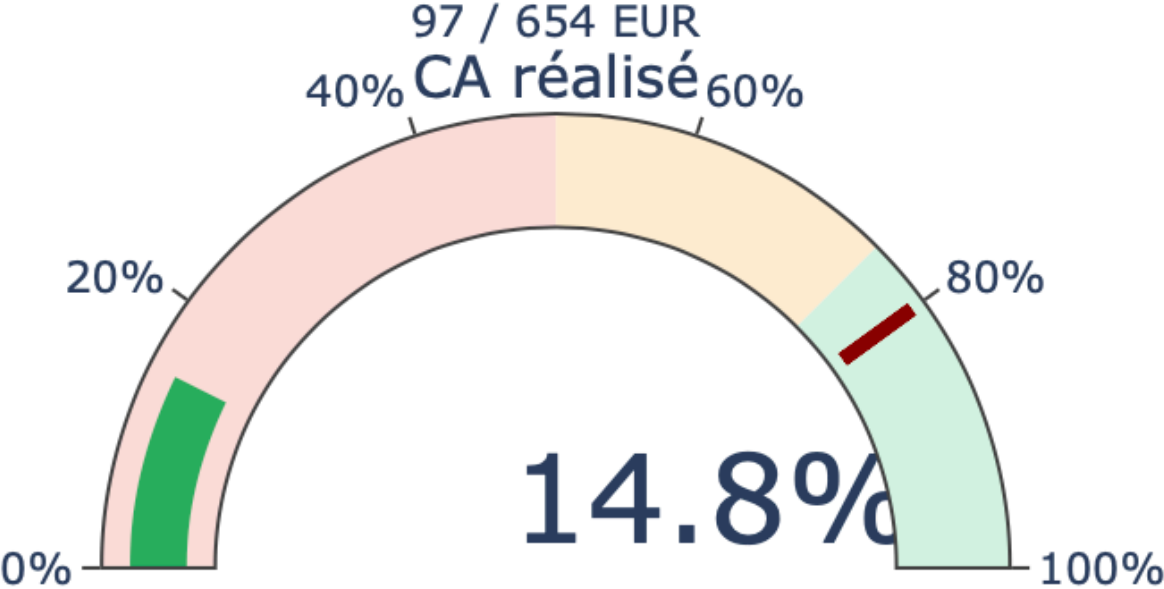
	Nom	C_max	Profit_done	Skill_gain	Cognitive_load	solution_valide	base_worker	need_level_if_solo	OK
0	solution	20	103.970774	0.5	43.807076	PASS	PASS	PASS	TRUE

Random Search



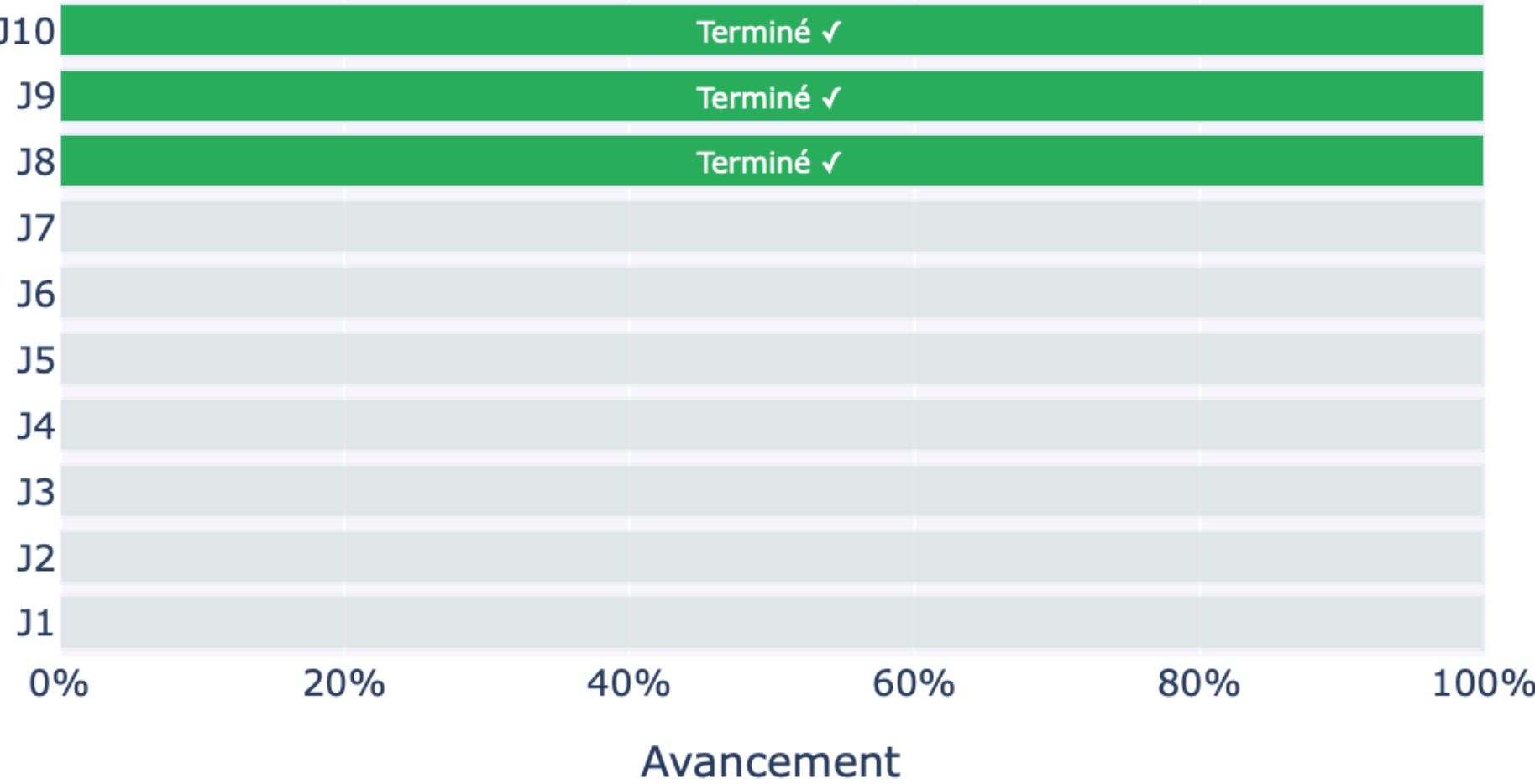
Planning par Job (makespan = 18.39)



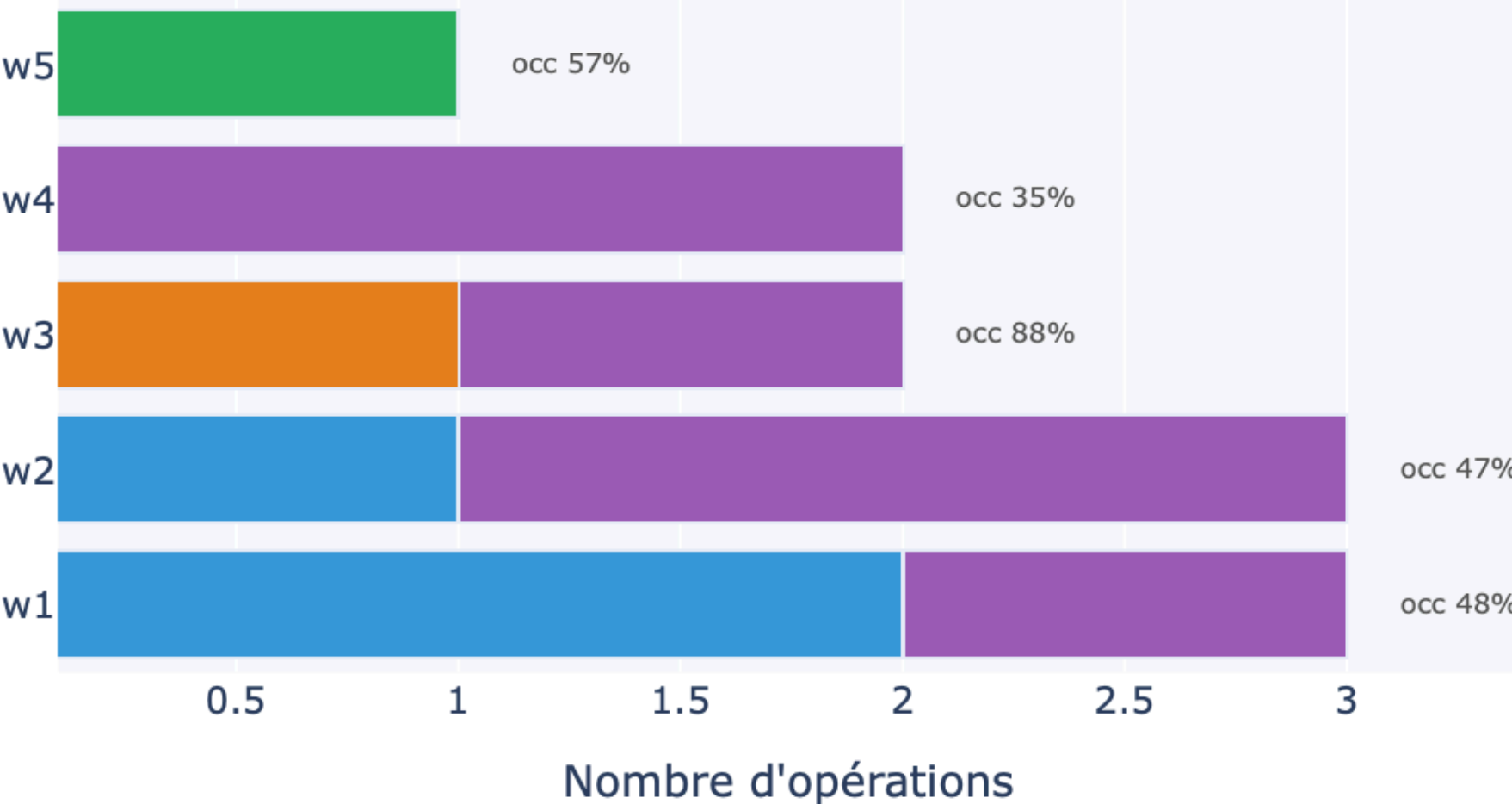


Random Search

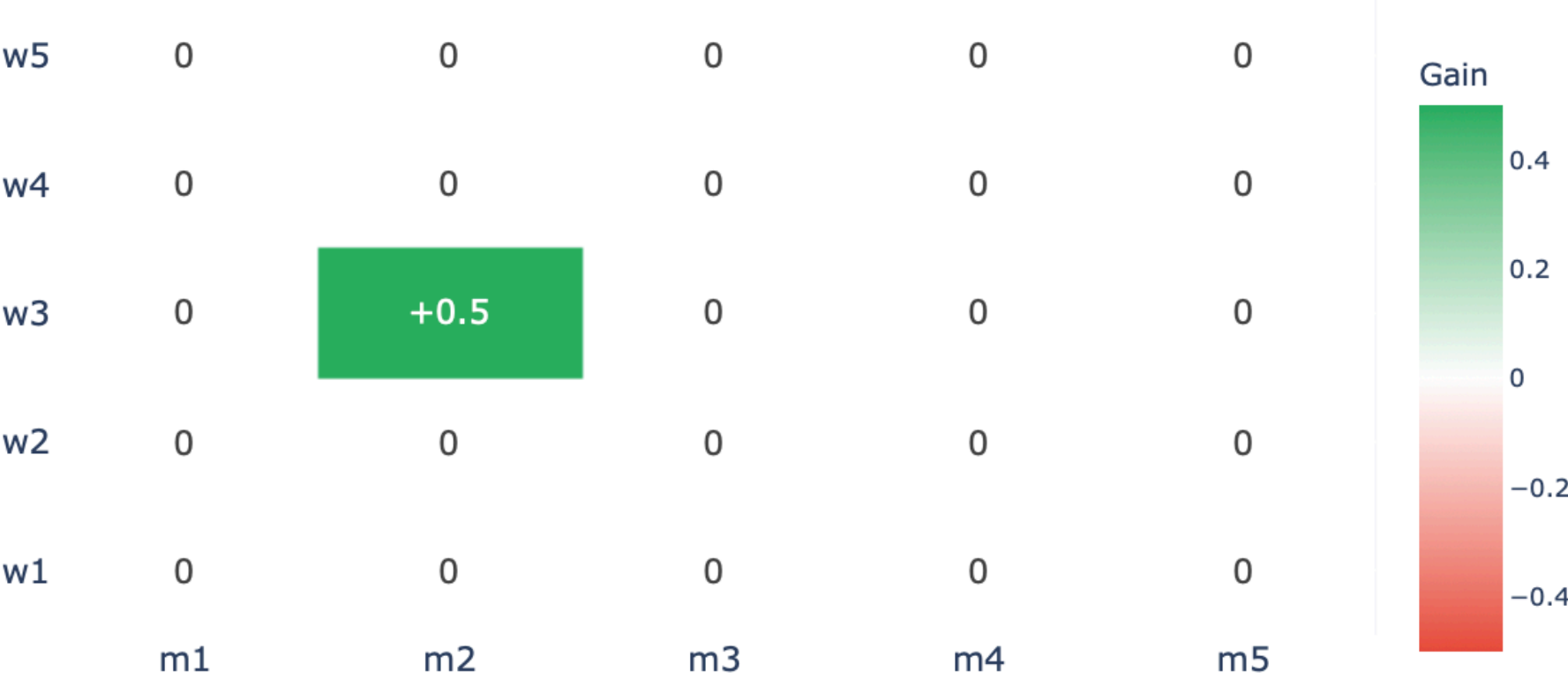
Avancement des jobs (terminés / en cours / non commencés)



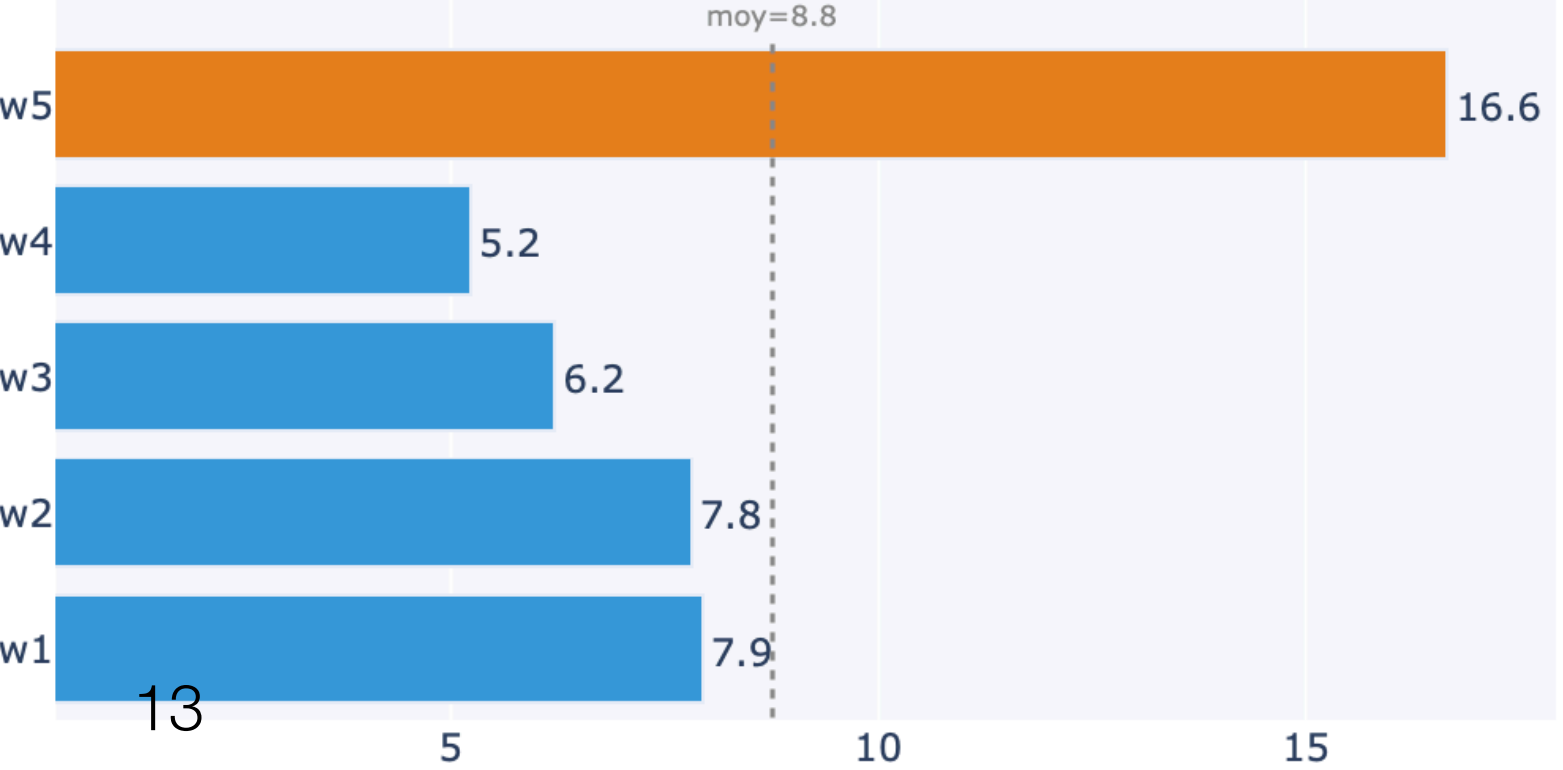
Occupation par mode d'exécution



Gains de compétences (workers × métiers)



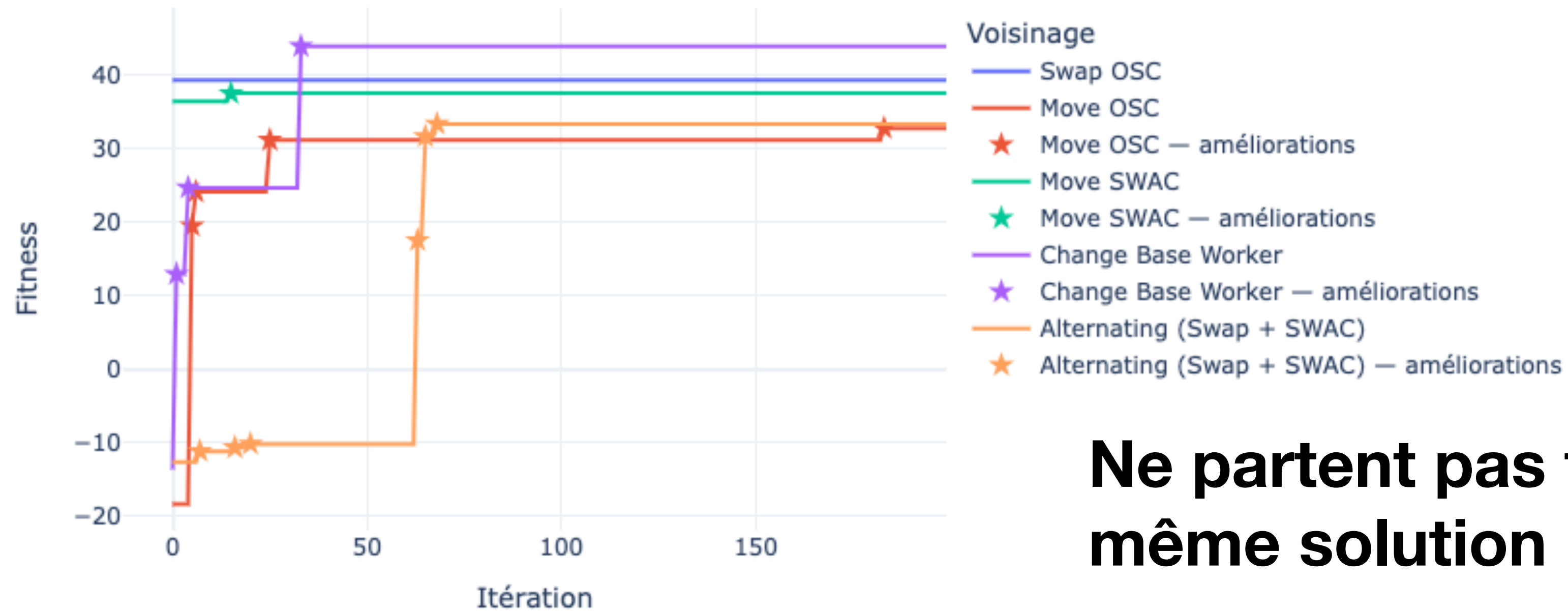
Charge cognitive par worker



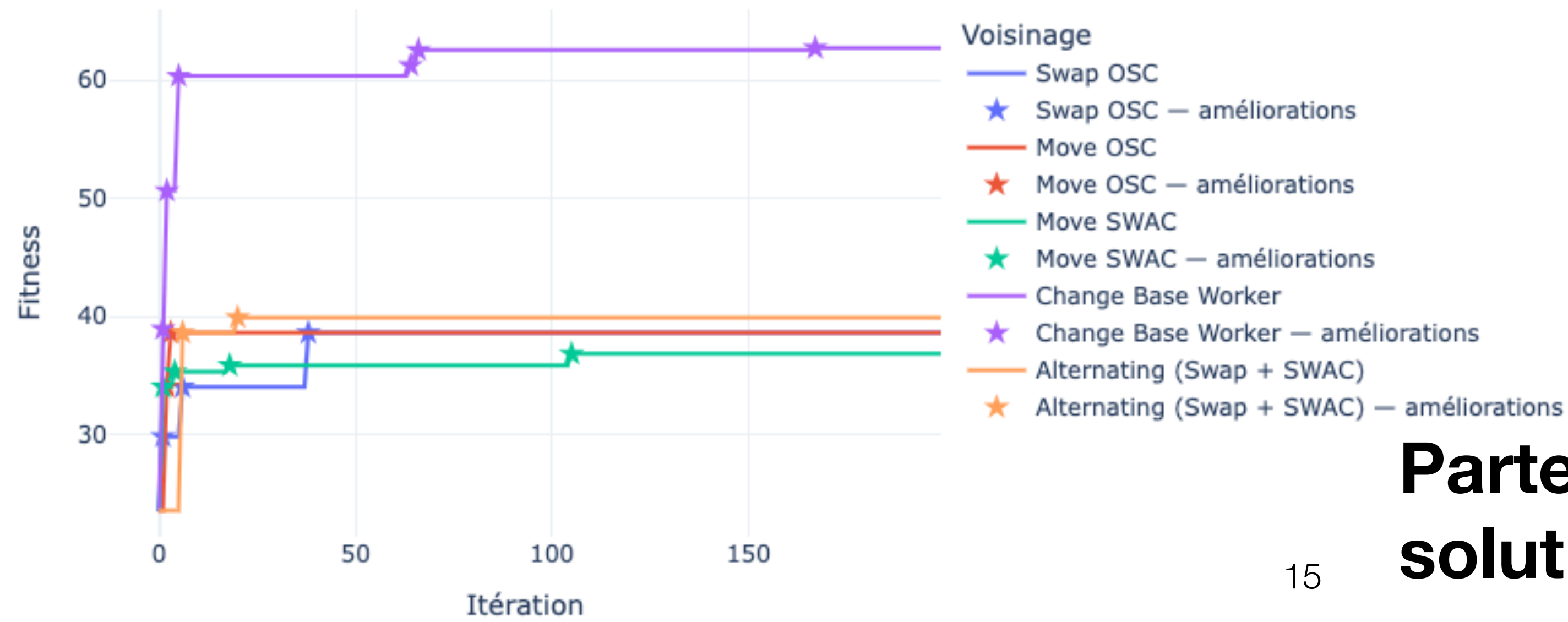
Local Search

Comparaison des voisinages — évolution de la fitness

Local Search



Comparaison des voisinages — évolution de la fitness



Meilleure solution trouvé pour chaque voisinage

Local Search

```
=====
Voisinage : Swap OSC | fitness : 38.6329
=====
```

	Nom	C_max	Profit_done	Skill_gain	Cognitive_load	solution_valide	base_worker	need_level_if_solo	OK
0	solution	20	76.308315	0.0	37.675438	PASS	PASS	PASS	TRUE

Swap_op : Génere n=10 voisins en permutant 2 opérations aléatoire

```
=====
Voisinage : Move OSC | fitness : 38.6329
=====
```

	Nom	C_max	Profit_done	Skill_gain	Cognitive_load	solution_valide	base_worker	need_level_if_solo	OK
0	solution	20	76.308315	0.0	37.675438	PASS	PASS	PASS	TRUE

Move_osc : Chosit une opération aléatoire et la déplace à toutes les positions possibles

```
=====
Voisinage : Move SWAC | fitness : 36.8516
=====
```

	Nom	C_max	Profit_done	Skill_gain	Cognitive_load	solution_valide	base_worker	need_level_if_solo	OK
0	solution	20	77.308315	1.0	41.456756	PASS	PASS	PASS	TRUE

Move_swac : choisit une opération aléatoire et change le worker 2 de cette opération parmi tout ceux possible

```
=====
Voisinage : Change Base Worker | fitness : 62.7176
=====
```

	Nom	C_max	Profit_done	Skill_gain	Cognitive_load	solution_valide	base_worker	need_level_if_solo	OK
0	solution	20	103.970774	0.5	41.753134	PASS	PASS	PASS	TRUE

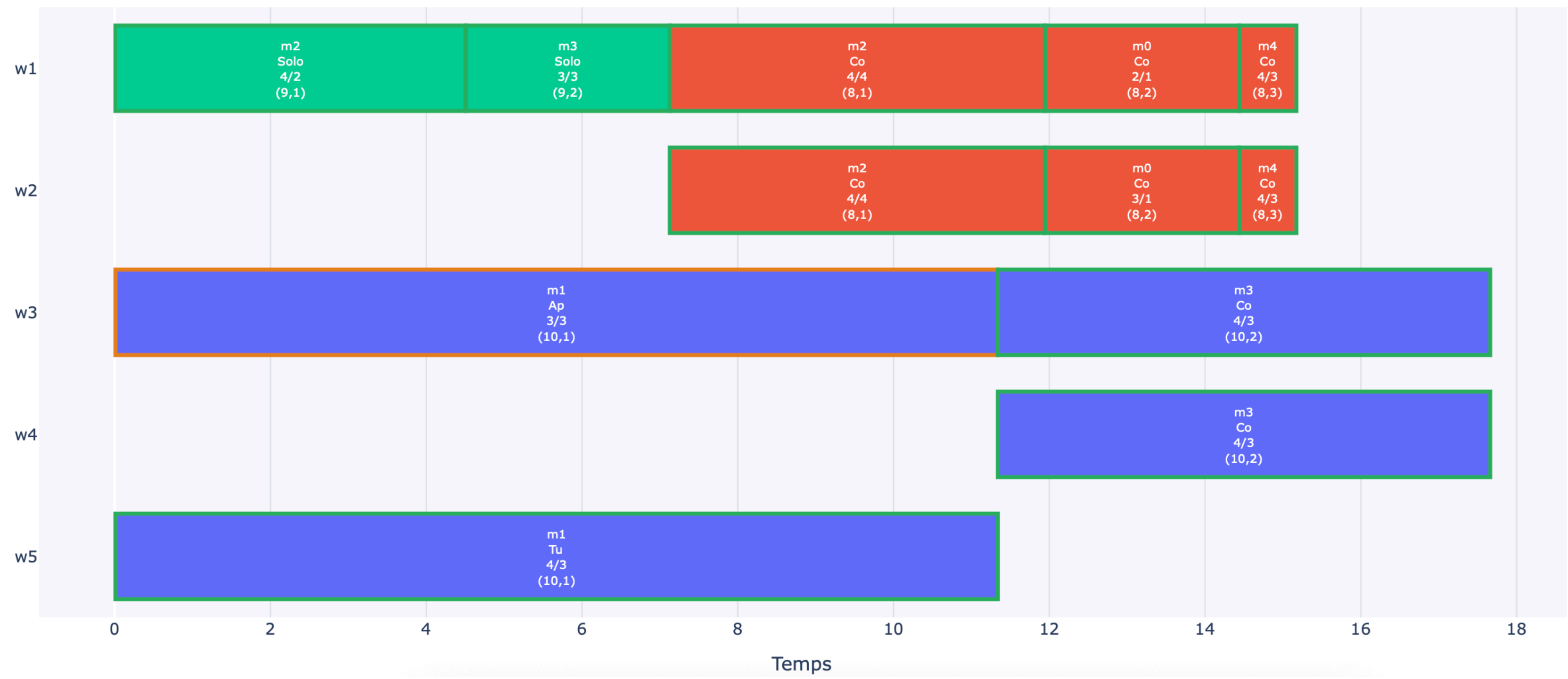
Base_worker : choisit une opération aléatoire et change le worker de base de celle ci

```
=====
Voisinage : Alternating (Swap + SWAC) | fitness : 39.9258
=====
```

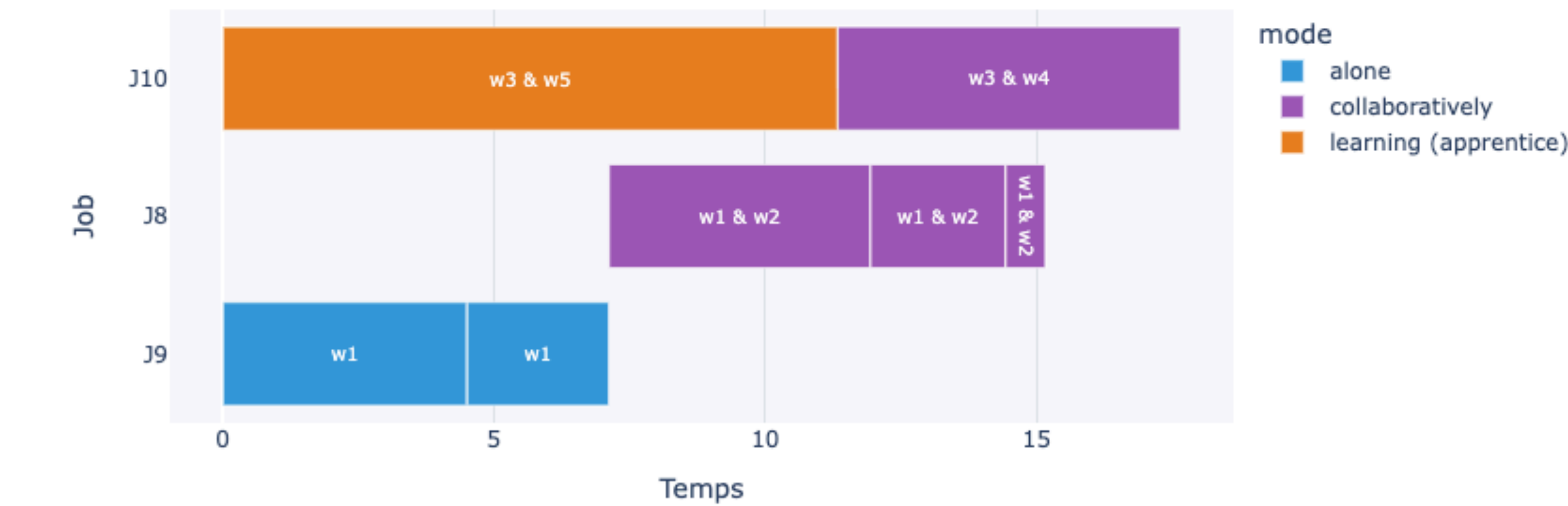
	Nom	C_max	Profit_done	Skill_gain	Cognitive_load	solution_valide	base_worker	need_level_if_solo	OK
0	solution	20	76.308315	0.0	36.38253	PASS	PASS	PASS	TRUE

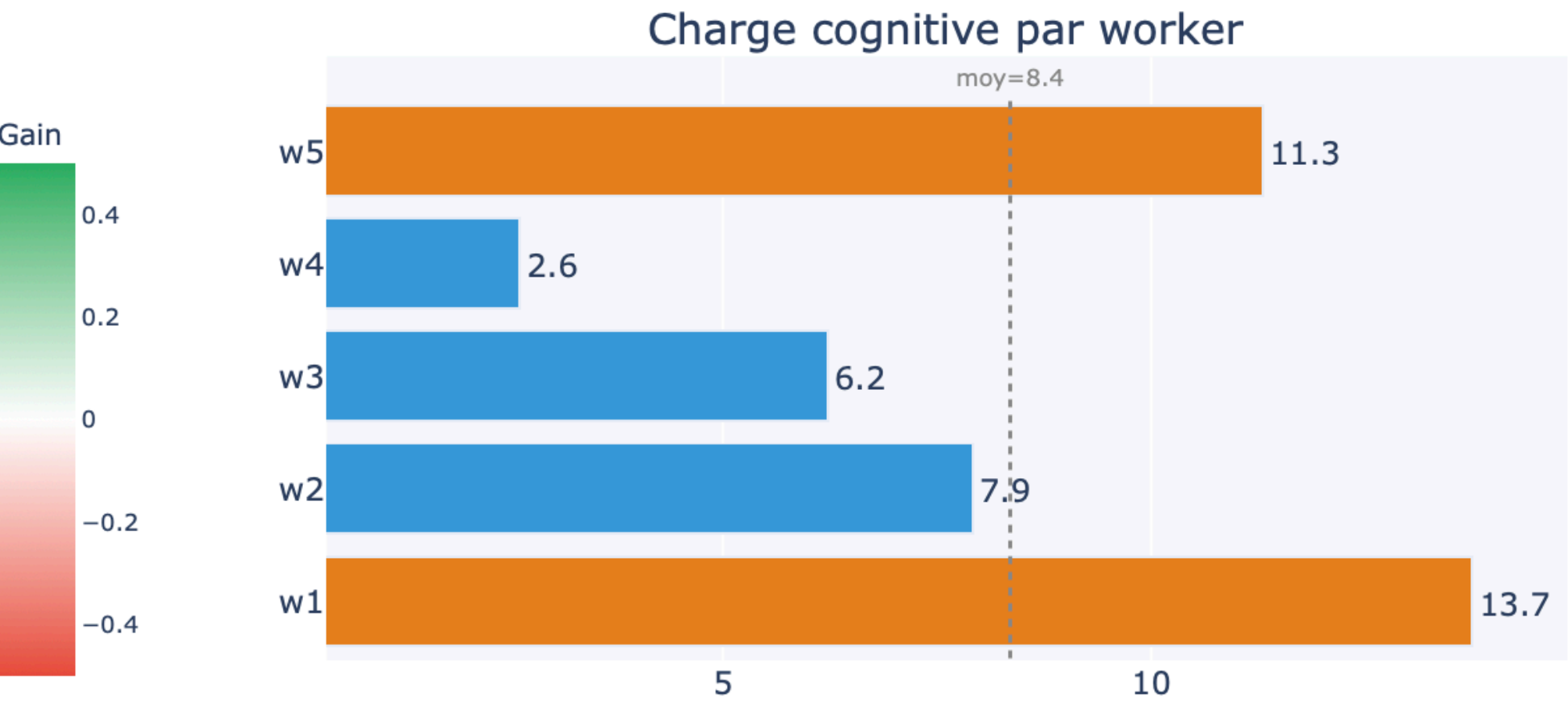
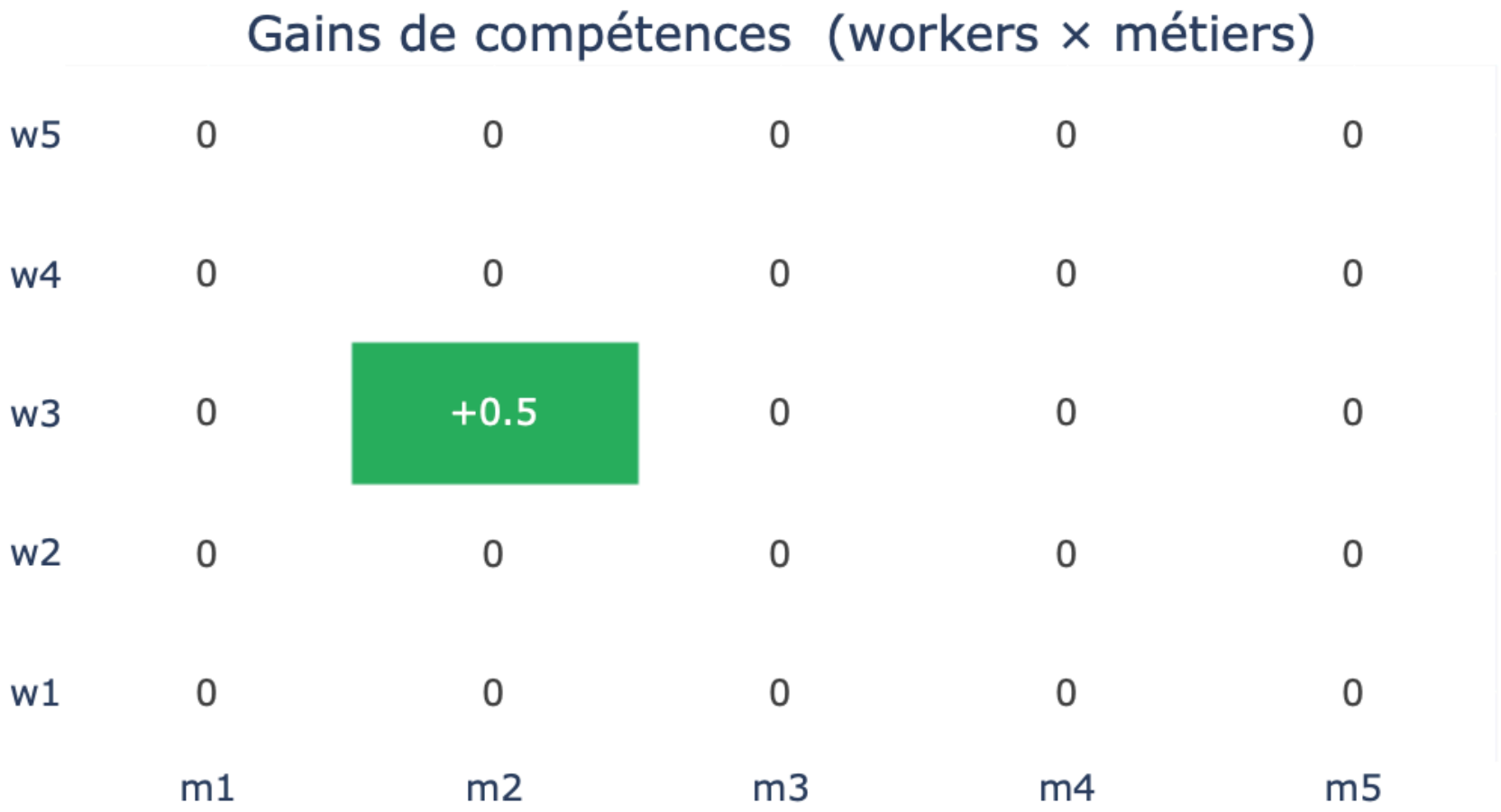
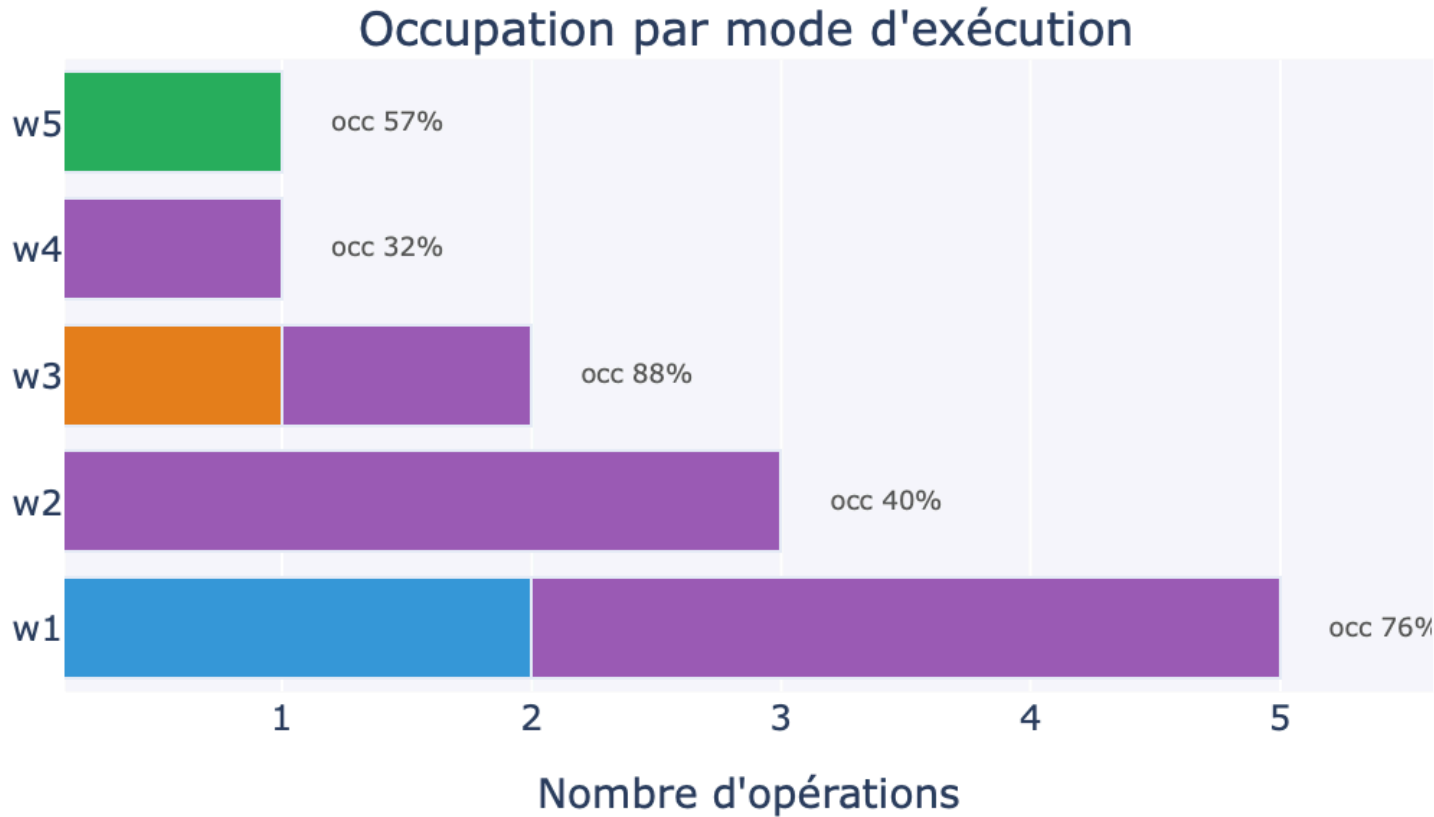
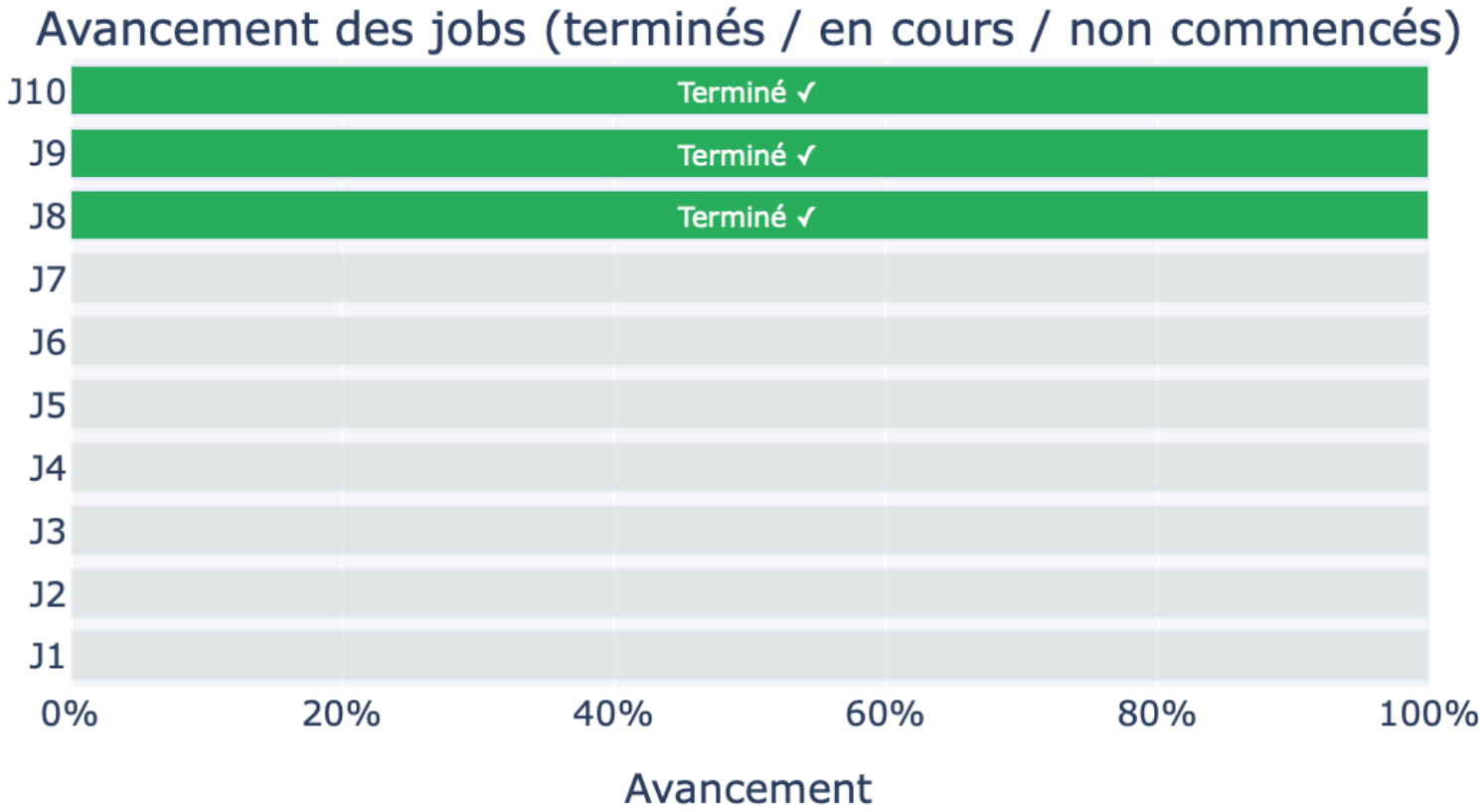
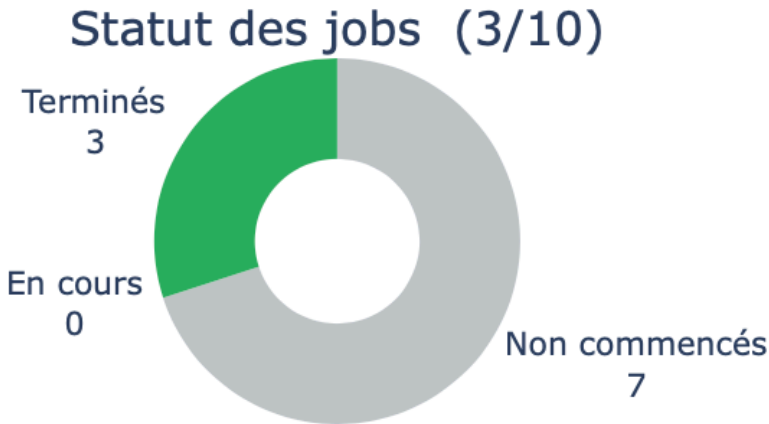
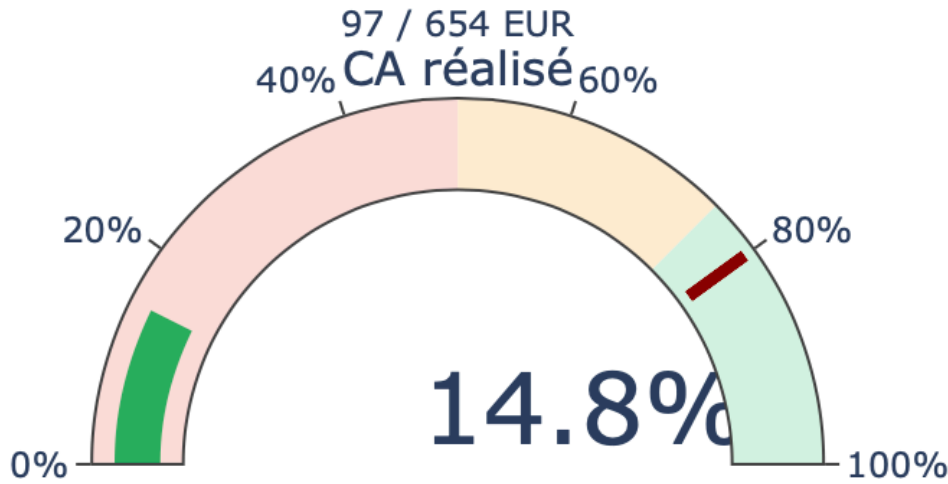
Comparaison entre **Base_worker** et **MILP**

Voisinage change base worker

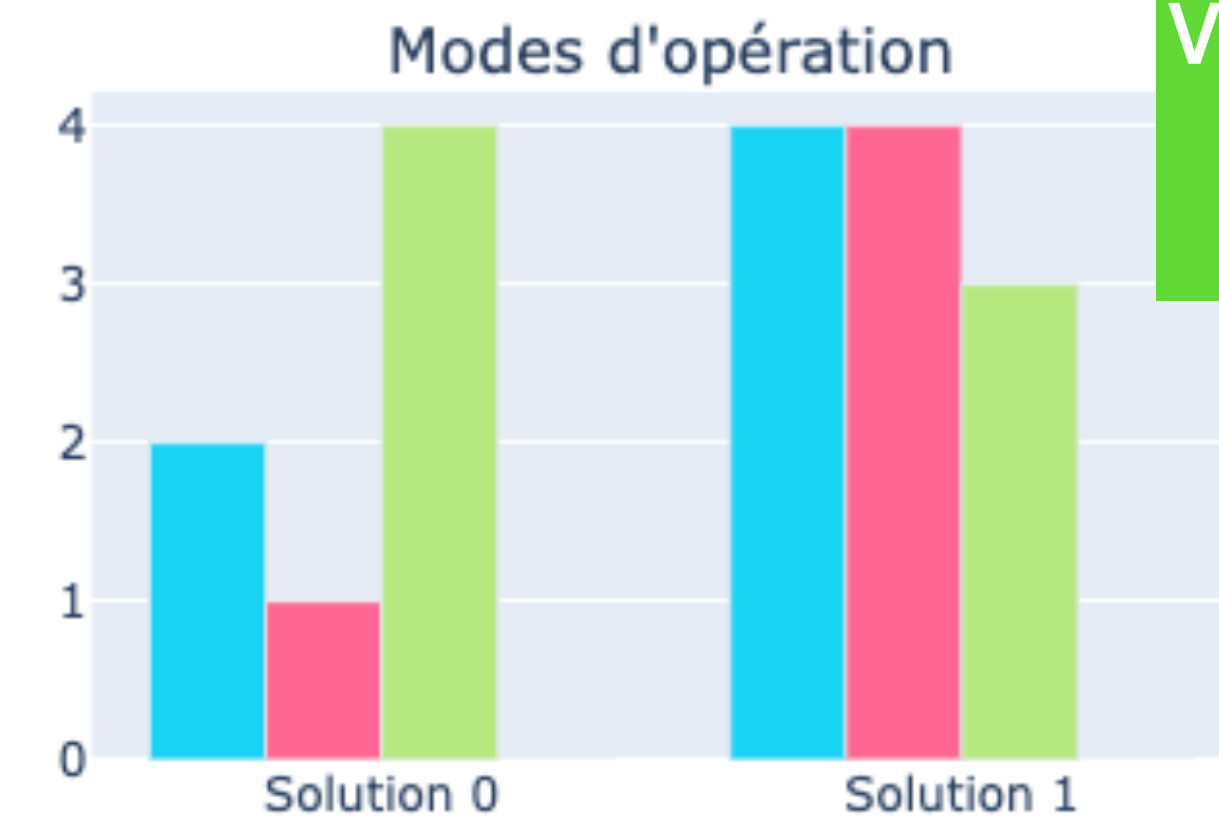
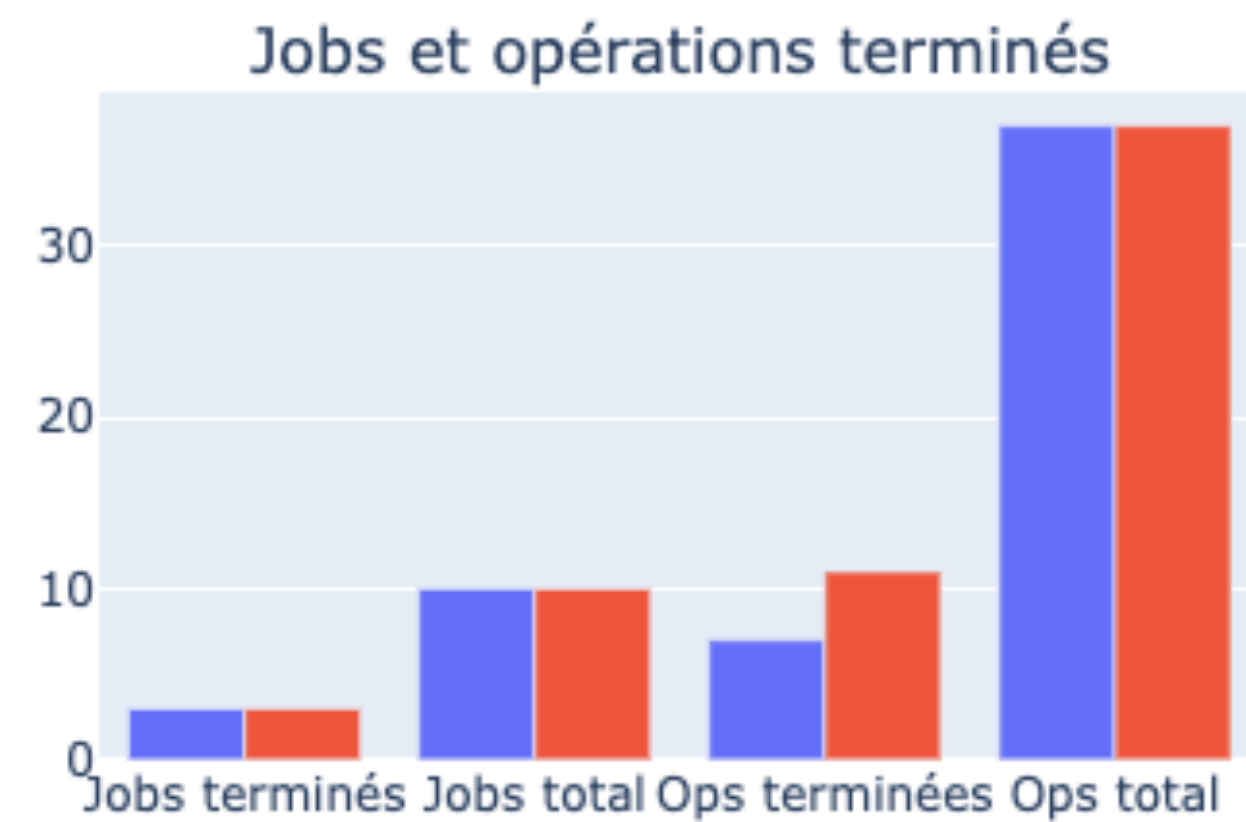


Planning par Job (makespan = 17.65)

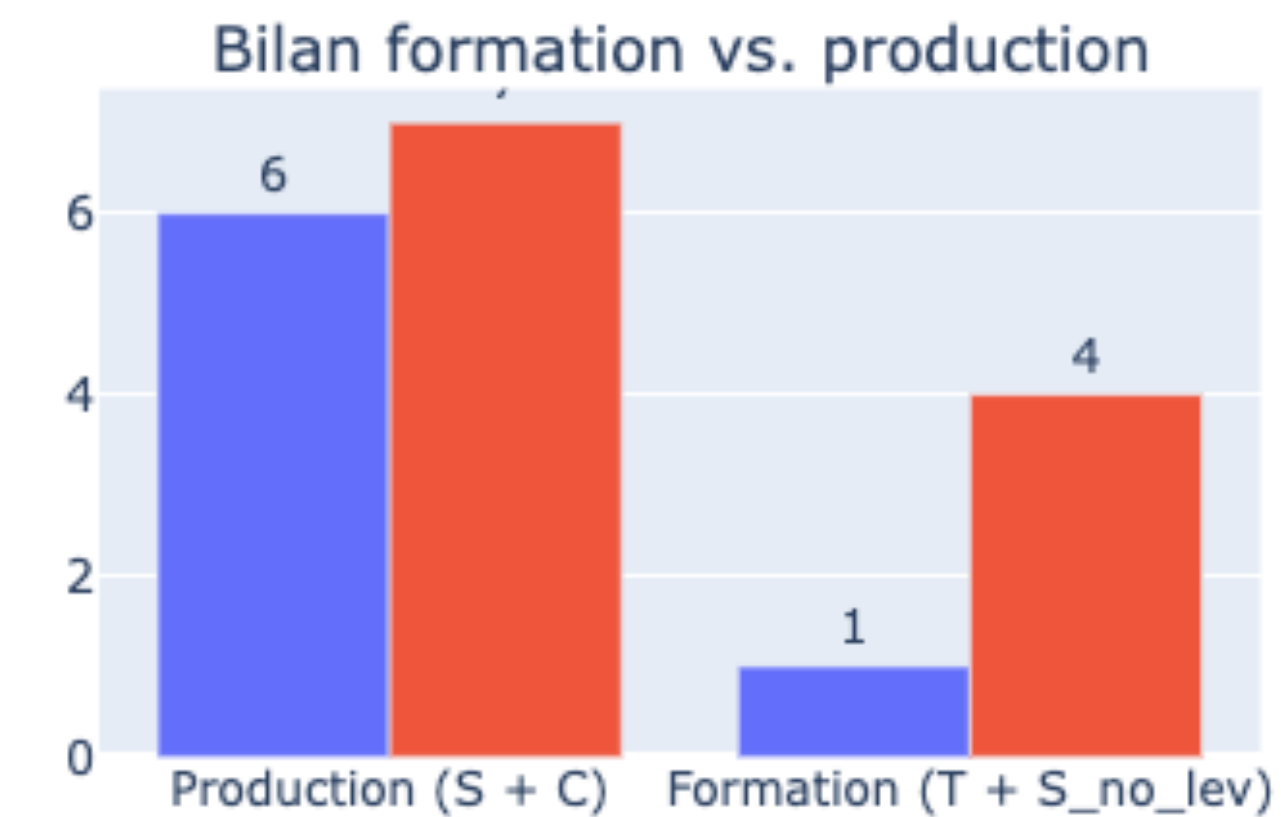




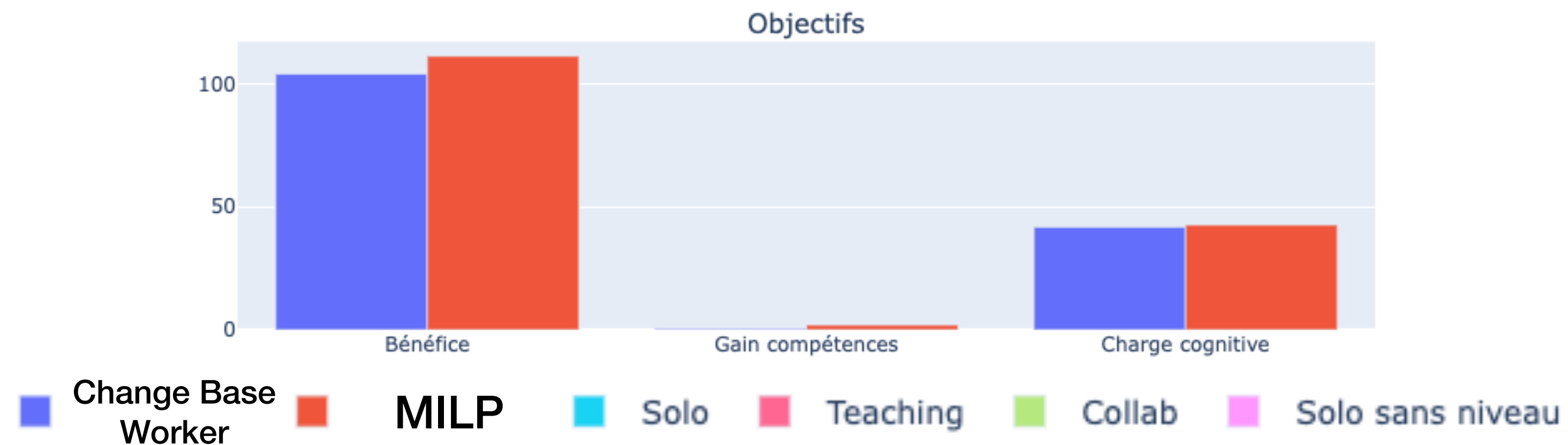
Voisinage change base worker
Vs
MILP



Comment le travail
est organisé



Opération réalisé
avec ou sans solo
sans level



Part du temps
investie dans la
montée en
compétence
(Formation) ou
Production directe
(solo ou collab)